

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

TERM WORK Theory and Practical Record

B.Tech Information Technology
Academic Year: 2026-27

Student Name: Aryan Sinha
Roll No.: 1
PRN: 2414110421

Table of Contents

This combined record is arranged so that each practical has approximately two pages of theory and preparation material. Verified code and execution evidence are referenced in the marked attachment areas and stored with the project files.

Assignment 1 - Association Between Dependent and Independent Variables

Assignment 2 - Clustering Using Unsupervised Learning

Assignment 3 - Dimensionality Reduction Using PCA

Assignment 4 - Naive Bayes Classification

Assignment 5 - KNN Classification

Assignment 6 - Linear Regression

Assignment 7 - SVM Algorithm for Regression

Assignment 8 - Regularization to Avoid Overfitting

Assignment 9 - Object Detection from CCTV Footage

Assignment 10 - Fake News Detection from Multimodal Data

Assignment 1: Association Between Dependent and Independent Variables

Aim: Identify and measure the statistical association between a dependent variable and relevant independent variables using correlation analysis and clear visualizations.

Objectives

- Differentiate dependent and independent variables.
- Measure direction and strength of linear association using Pearson correlation.
- Use scatter plots and a correlation heatmap to visualize relationships.
- Interpret association carefully without treating correlation as proof of causation.

Key Definitions

Term	Definition
Dependent variable	Outcome or response variable being studied.
Independent variable	Explanatory feature whose relationship with the outcome is examined.
Correlation	Numerical description of the direction and strength of association between variables.
Pearson correlation (r)	Linear association measure ranging from -1 to +1.
p-value	Evidence used with a statistical test to assess whether an observed correlation differs from zero under stated assumptions.

Theory / Working Principle

- Correlation is positive when larger values of one variable tend to occur with larger values of the other; it is negative when they tend to move in opposite directions.
- Pearson correlation is appropriate for numerical variables when a linear relationship is the main interest.
- A large absolute correlation does not establish a causal relationship; plotting the data helps reveal outliers or nonlinear patterns.

Implementation note: A small local student-performance CSV was used for reproducible Pearson correlation and visualization.

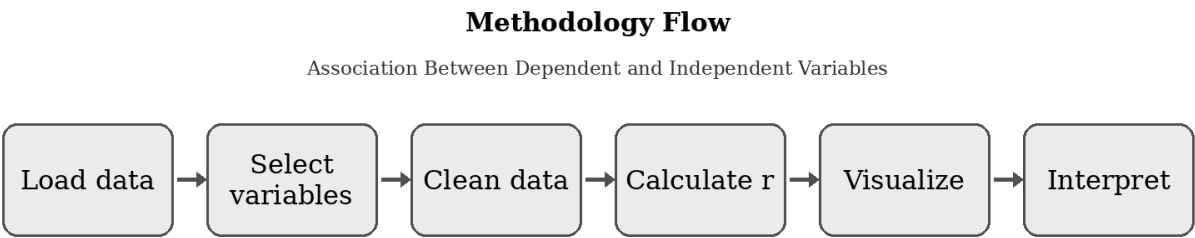


Figure 1. Methodology flow for Assignment 1.

Tools and Technologies

Tool / Technology	Role in the practical
Pandas	Load CSV data and clean selected columns
SciPy	Calculate Pearson r and p-value
Seaborn	Correlation heatmap
Matplotlib	Scatter plots
VS Code + Python	Execution

Procedure

1. Read dataset and inspect columns.
2. Choose dependent and independent numerical variables.
3. Handle missing or invalid values.
4. Calculate Pearson r and p-value for important pairs.
5. Create scatter plots and a correlation heatmap.
6. Interpret direction, strength and limitations.

Evaluation and Expected Output

Important outputs / measures: correlation coefficient r, p-value, scatter plot, correlation heatmap.

The final output should provide visible evidence that the algorithm or statistical method was executed successfully. The observation must be based on the actual values produced during execution, not on assumed results.

Result:

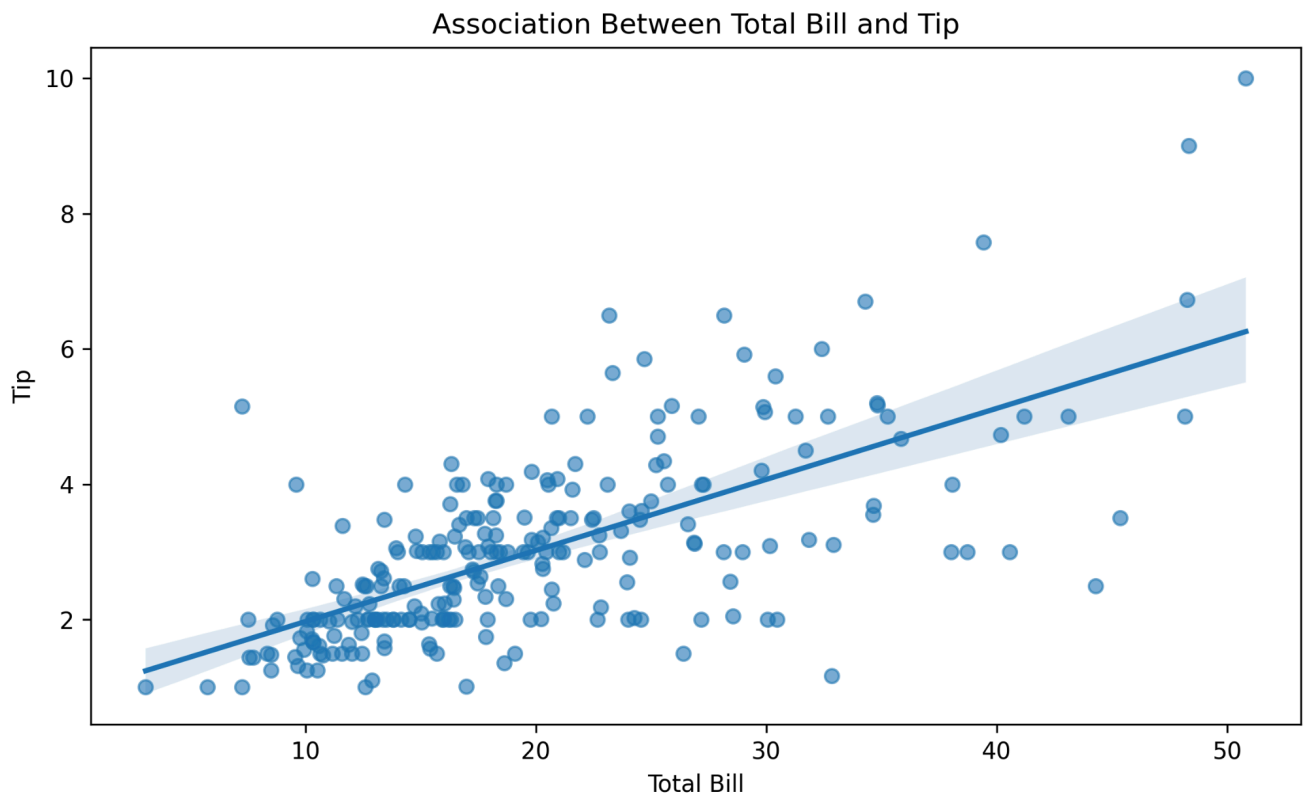
The association between the independent variable `total_bill` and the dependent variable `tip` was analyzed using Pearson correlation. The calculated correlation coefficient was 0.6757, indicating a positive linear association between the two variables. The scatter plot with a regression line also shows that the tip generally increases as the total bill increases. The correlation heatmap further represents the relationship among the numerical variables in the dataset.

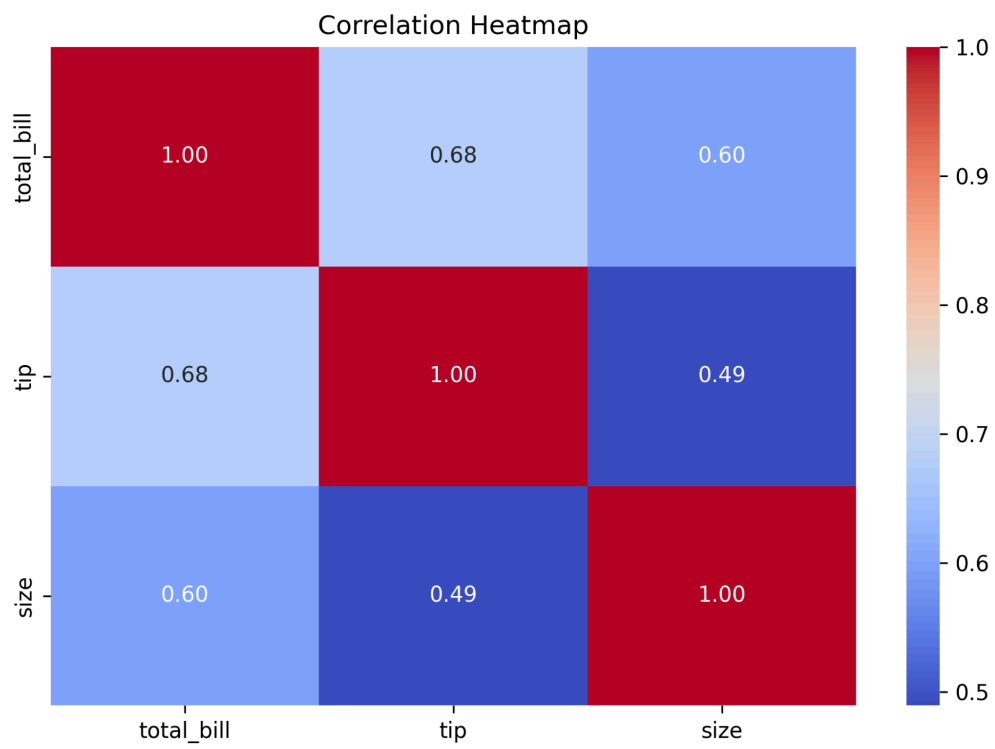
Hence, **`total_bill` and `tip` have a positive association**, with higher total bills generally corresponding to higher tips.

Code Attachment: assignment_01/code/assignment_01_association.py

```
assignment_01.py X
Practical_1 > assignment_01.py > ...
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 # -----
7 # Assignment 1: Association Between Variables
8 # -----
9
10 print("-" * 55)
11 print("ASSIGNMENT 1: ASSOCIATION BETWEEN VARIABLES")
12 print("-" * 55)
13
14 # Load public dataset
15 df = sns.load_dataset("tips")
16
17 # Dataset information
18 print("\n--- Dataset Preview ---")
19 print(df.head())
20
21 print("\n--- Dataset Shape ---")
22 print(df.shape)
23
24 print("\n--- Dataset Information ---")
25 print(df.info())
26
27 # Select independent and dependent variables
28 X = df["total_bill"]
29 Y = df["tip"]
30
31 # Calculate Pearson correlation
32 correlation = X.corr(Y)
33
34 print("\n--- Variable Association ---")
35 print("Independent Variable : total_bill")
36 print("Dependent Variable : tip")
37 print(f"Pearson Correlation : {correlation:.4f}")
38
39 if correlation > 0:
40     print("Association: Positive")
41 elif correlation < 0:
42     print("Association: Negative")
43 else:
44     print("Association: No linear association")
45
46 # -----
47
48 assignment_01.py X
Practical_1 > assignment_01.py > ...
44 else:
45     print("Association: No linear association")
46
47 # -----
48 # Scatter Plot with Regression Line
49 # -----
50 plt.figure(figsize=(8, 5))
51
52 sns.regplot(
53     x="total_bill",
54     y="tip",
55     data=df,
56     scatter_kws={"alpha": 0.6}
57 )
58
59 plt.title("Association Between Total Bill and Tip")
60 plt.xlabel("Total Bill")
61 plt.ylabel("Tip")
62 plt.tight_layout()
63
64 plt.savefig("association_scatter_plot.png", dpi=300)
65 plt.show()
66
67 # -----
68 # Correlation Heatmap
69 # -----
70
71 numeric_data = df.select_dtypes(include=np.number)
72
73 plt.figure(figsize=(7, 5))
74
75 sns.heatmap(
76     numeric_data.corr(),
77     annot=True,
78     cmap="coolwarm",
79     fmt=".2f"
80 )
81
82 plt.title("Correlation Heatmap")
83 plt.tight_layout()
84
85 plt.savefig("correlation_heatmap.png", dpi=300)
86 plt.show()
87
88 print("\n--- Execution Completed Successfully ---")
```

Output / Screenshot Attachment: assignment_01/screenshots/execution_capture.png





```
=====
ASSIGNMENT 1: ASSOCIATION BETWEEN VARIABLES
=====
```

```
--- Dataset Preview ---
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

```
--- Dataset Shape ---
```

```
(244, 7)
```

```
--- Dataset Information ---
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 244 entries, 0 to 243
```

```
Data columns (total 7 columns):
```

#	Column	Non-Null Count	Dtype
0	total_bill	244 non-null	float64
1	tip	244 non-null	float64
2	sex	244 non-null	category
3	smoker	244 non-null	category
4	day	244 non-null	category
5	time	244 non-null	category
6	size	244 non-null	int64

```
dtypes: category(4), float64(2), int64(1)
```

```
memory usage: 7.4 KB
```

```
None
```

```
--- Variable Association ---
```

```
Independent Variable : total_bill
```

```
Dependent Variable   : tip
```

```
Pearson Correlation   : 0.6757
```

```
Association: Positive
```

Assignment 2: Clustering Using Unsupervised Learning

Aim: Apply K-Means clustering to divide observations into groups with similar feature patterns without using predefined class labels.

Objectives

- Understand unsupervised learning and cluster formation.
- Scale features before distance-based clustering.
- Use the Elbow method and silhouette score to guide the choice of K.
- Summarize the characteristics of the discovered clusters.

Key Definitions

Term	Definition
Unsupervised learning	Learning patterns without a target label supplied during training.
Cluster	Group of observations that are more similar to each other than to observations in other groups.
K-Means	Iterative algorithm that assigns observations to the nearest centroid and repeatedly updates centroids.
Centroid	Mean feature position representing a cluster.
Silhouette score	Measure comparing within-cluster similarity with separation from another cluster.

Theory / Working Principle

- K-Means minimizes the within-cluster sum of squared distances from observations to their assigned centroids.
- Scaling is usually required because Euclidean distance is affected by feature magnitude.
- The number of clusters is selected as a practical modeling choice using the Elbow curve and silhouette score.

Implementation note: A small local customer-segmentation CSV with annual income and spending score was used; features were standardized before K-Means.

Methodology Flow

Clustering Using Unsupervised Learning

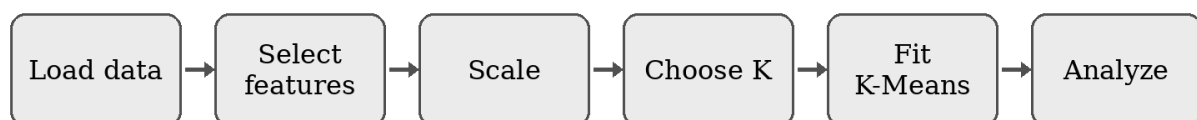


Figure 2. Methodology flow for Assignment 2.

Tools and Technologies

Tool / Technology	Role in the practical
Pandas	Load and inspect dataset
StandardScaler	Scale numerical features
KMeans	Fit clusters
Matplotlib / Seaborn	Elbow and cluster plots
Scikit-learn metrics	Silhouette score

Procedure

1. Load and inspect the dataset.
2. Select meaningful numerical features.
3. Standardize the selected features.
4. Evaluate several K values with inertia and silhouette score.
5. Fit final K-Means and assign cluster labels.
6. Plot clusters and summarize cluster sizes/means.

Evaluation and Expected Output

Important outputs / measures: inertia, silhouette score, cluster size, feature means.

The final output should provide visible evidence that the algorithm or statistical method was executed successfully. The observation must be based on the actual values produced during execution, not on assumed results.

Result / Observation

K-Means clustering was successfully applied to the **Mall Customers dataset** using **Annual Income** and **Spending Score** as clustering features. After evaluating different values of K using inertia and silhouette scores, **K = 5** was selected. The resulting model achieved a **silhouette score of 0.5547** and divided the 200 customers into five clusters with sizes of **81, 39, 22, 35, and 23**. The cluster means indicated distinct customer groups based on income and spending behavior, demonstrating the usefulness of K-Means for customer segmentation.

Code Attachment: assignment_02/code/assignment_02_clustering.py

```
py Mall_Customers.csv assignment_02_clustering.py X ... assignment_02_clustering.py X ... assignment_02_clustering.py X ... assignment_02_clustering.py X ... BLACKBOX ...

Practical_2 > assignment_02_clustering.py > ...
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.cluster import KMeans
8 from sklearn.metrics import silhouette_score
9
10 print("=" * 60)
11 print("ASSIGNMENT 2: K-MEANS CLUSTERING")
12 print("=" * 60)
13
14 # Load dataset
15 df = pd.read_csv("Mall_Customers.csv")
16
17 print("\n--- Dataset Preview ---")
18 print(df.head())
19
20 print("\n--- Dataset Shape ---")
21 print(df.shape)
22
23 print("\n--- Missing Values ---")
24 print(df.isnull().sum())
25
26 # Select features
27 features = ["Annual Income (k$)", "Spending Score (1-100)"]
28 X = df[features]
29
30 # Standardize
31 scaler = StandardScaler()
32 X_scaled = scaler.fit_transform(X)
33
34 print("\n--- Selected Features ---")
35 print(features)
36
37 # Evaluate different K values
38 inertia = []
39 silhouette_scores = []
40 k_values = range(2, 11)
41
42 for k in k_values:
43     model = KMeans(n_clusters=k, random_state=42, n_init=10)
44     labels = model.fit_predict(X_scaled)
45     inertia.append(model.inertia_)
46     silhouette_scores.append(silhouette_score(X_scaled, labels))
47
48 print("\n--- K Evaluation ---")
49 for k, i, s in zip(k_values, inertia, silhouette_scores):
50     print(f"K={k}: Inertia={i:.4f}, Silhouette Score={s:.4f}")
51
52 # Plot Elbow Method
53 plt.figure(figsize=(8, 5))
54 plt.plot(k_values, inertia, marker="o")
55 plt.xlabel("Number of Clusters (K)")
56 plt.ylabel("Inertia")
57 plt.title("Elbow Method for Optimal K")
58 plt.grid(True)
59 plt.tight_layout()
60 plt.savefig("elbow_plot.png", dpi=300)
61 plt.show()
62
63 # Plot Silhouette Scores
64 plt.figure(figsize=(8, 5))
65 plt.plot(k_values, silhouette_scores, marker="o")
66 plt.xlabel("Number of Clusters (K)")
67 plt.ylabel("Silhouette Score")
68 plt.title("Silhouette Score for Different K Values")
69 plt.grid(True)
70 plt.tight_layout()
71 plt.savefig("silhouette_plot.png", dpi=300)
72 plt.show()
73
74 # Final K-Means model
75 optimal_k = 5
76
77 kmeans = KMeans(
78     n_clusters=optimal_k,
79     random_state=42,
80     n_init=10
81 )
82
83 df["Cluster"] = kmeans.fit_predict(X_scaled)
84 final_silhouette = silhouette_score(X_scaled, df["Cluster"])
85
86 print("\n--- Final K-Means Results ---")
87 print(f"Selected K : {optimal_k}")
88 print(f"Silhouette Score : {final_silhouette:.4f}")
89
90 print("\n--- Cluster Sizes ---")
91
92 Practical_2 > assignment_02_clustering.py > ...
87
88 print("\n--- Final K-Means Results ---")
89 print(f"Selected K : {optimal_k}")
90 print(f"Silhouette Score : {final_silhouette:.4f}")
91
92 print("\n--- Cluster Sizes ---")
93 print(df["Cluster"].value_counts().sort_index())
94
95 print("\n--- Cluster Means ---")
96 print(
97     df.groupby("Cluster")[features]
98     .mean()
99     .round(2)
100 )
101
102 # Cluster visualization
103 plt.figure(figsize=(9, 6))
104 sns.scatterplot(
105     data=df,
106     x="Annual Income (k$)",
107     y="Spending Score (1-100)",
108     hue="Cluster",
109     palette="deep",
110     s=80
111 )
112
113 centers_original = scaler.inverse_transform(kmeans.cluster_centers_)
114
115 plt.scatter(
116     centers_original[:, 0],
117     centers_original[:, 1],
118     marker="x",
119     s=200,
120     label="Centroids"
121 )
122
123 plt.title("Customer Segmentation using K-Means")
124 plt.xlabel("Annual Income (k$)")
125 plt.ylabel("Spending Score (1-100)")
126 plt.legend()
127 plt.tight_layout()
128 plt.savefig("cluster_plot.png", dpi=300)
129 plt.show()
130
131 print("\n--- Execution Completed Successfully ---")
```

Output / Screenshot Attachment: assignment_02/screenshots/execution_capture.png

```
=====
ASSIGNMENT 2: K-MEANS CLUSTERING
=====
```

```
--- Dataset Preview ---
```

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
--- Dataset Shape ---
```

```
(200, 5)
```

```
--- Missing Values ---
```

```
CustomerID      0
Genre            0
Age             0
Annual Income (k$)  0
Spending Score (1-100)  0
dtype: int64
```

```
--- Selected Features ---
```

```
['Annual Income (k$)', 'Spending Score (1-100)']
```

```
--- K Evaluation ---
```

```
K=2: Inertia=269.6910, Silhouette Score=0.3213
K=3: Inertia=157.7040, Silhouette Score=0.4666
K=4: Inertia=108.9213, Silhouette Score=0.4939
K=5: Inertia=65.5684, Silhouette Score=0.5547
K=6: Inertia=55.0573, Silhouette Score=0.5399
K=7: Inertia=44.8648, Silhouette Score=0.5281
K=8: Inertia=37.2282, Silhouette Score=0.4552
K=9: Inertia=32.3923, Silhouette Score=0.4571
K=10: Inertia=29.9819, Silhouette Score=0.4432
```

```

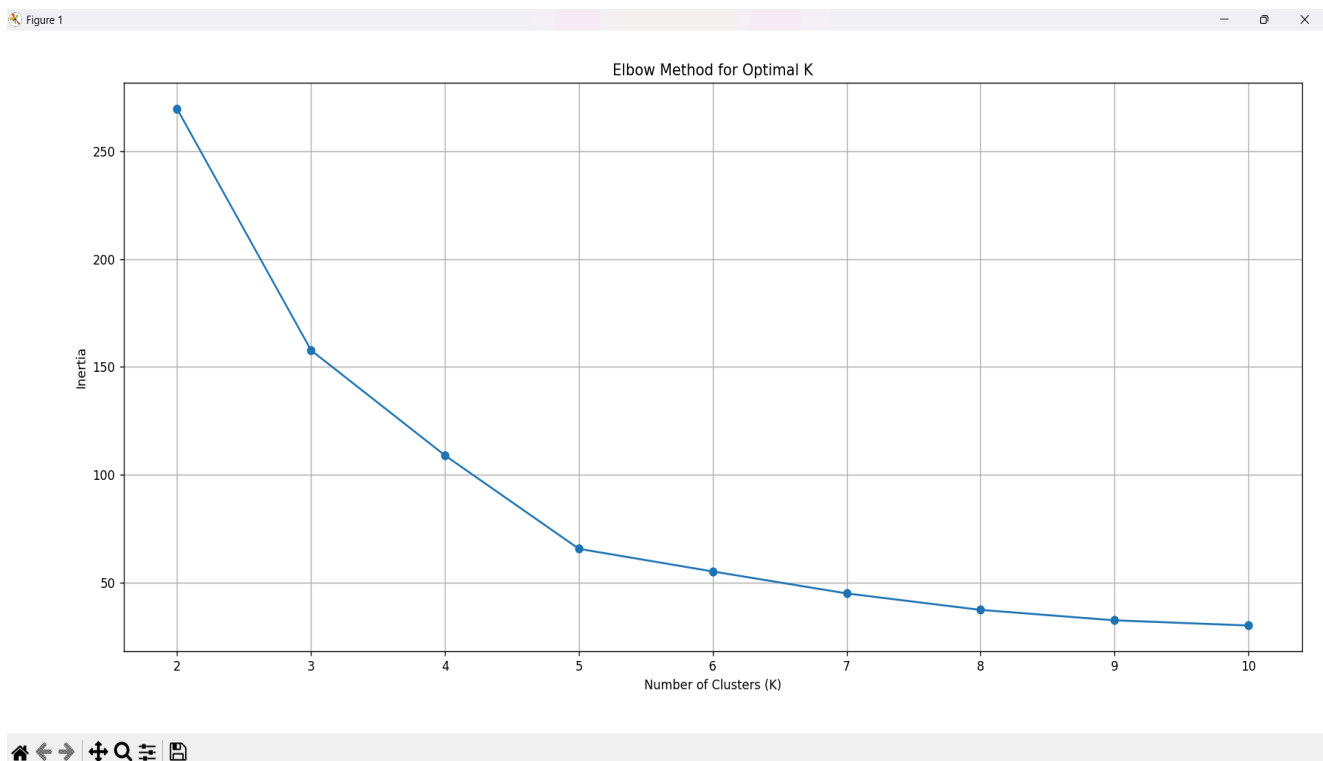
--- Final K-Means Results ---
Selected K      : 5
Silhouette Score : 0.5547

--- Cluster Sizes ---
Cluster
0      81
1      39
2      22
3      35
4      23
Name: count, dtype: int64

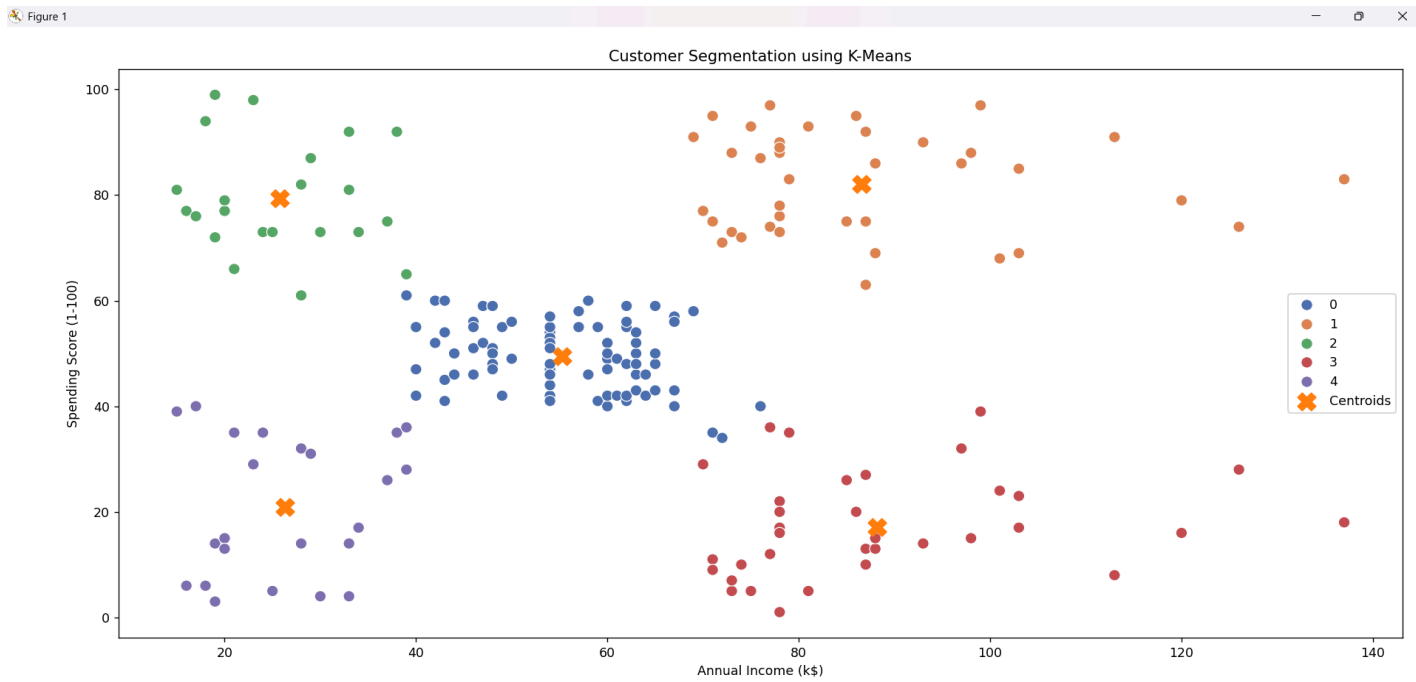
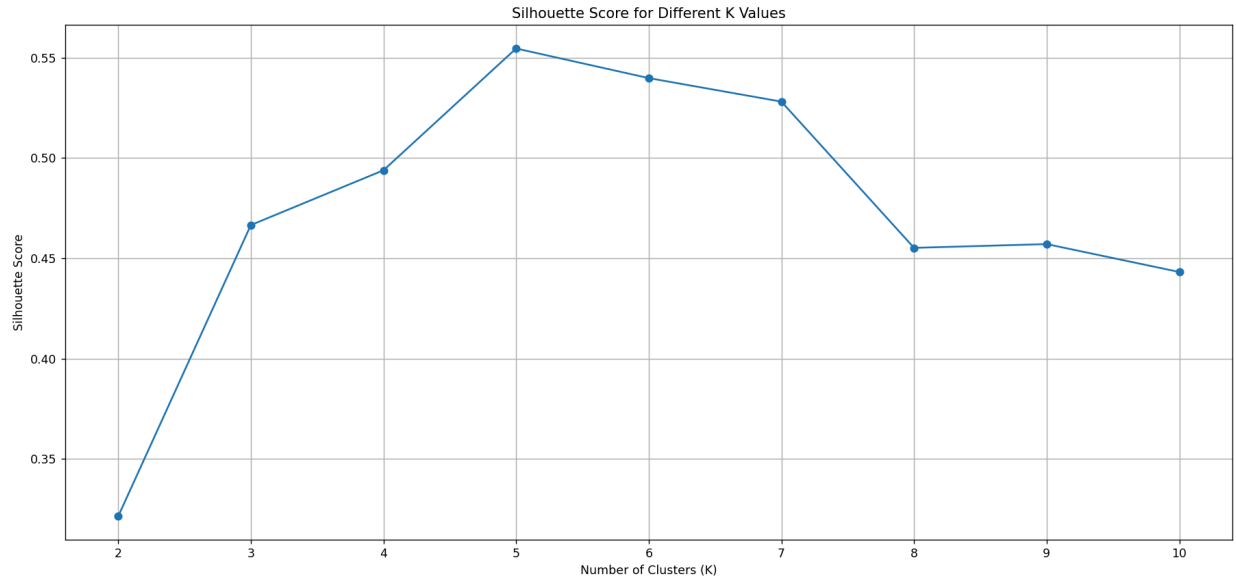
--- Cluster Means ---
          Annual Income (k$)  Spending Score (1-100)
Cluster
0                55.30             49.52
1                86.54             82.13
2                25.73             79.36
3                88.20             17.11
4                26.30             20.91

--- Execution Completed Successfully ---

```



K Evaluation Graph



Assignment 3: Dimensionality Reduction Using PCA

Aim: Reduce the number of numerical features with Principal Component Analysis while retaining a practical amount of the original variance.

Objectives

- Understand why dimensionality reduction is useful.
- Standardize features before PCA when their scales differ.
- Transform correlated original features into principal components.
- Use explained variance to choose a compact representation.

Key Definitions

Term	Definition
Dimensionality reduction	Representing data using fewer variables while preserving useful information.
Principal component	New orthogonal direction capturing a portion of the data variance.
Explained variance ratio	Proportion of total variance captured by a component.
Covariance matrix	Matrix describing how variables vary together and used in PCA.
Standardization	Rescaling features to comparable mean and spread before variance-sensitive analysis.

Theory / Working Principle

- PCA finds orthogonal directions that successively capture maximum variance.
- Each principal component is a linear combination of the original features.
- The first components generally capture the greatest share of variance, allowing visualization and feature compression.

Implementation note: The scikit-learn built-in Wine dataset was used because it contains 13 numerical features suitable for PCA.

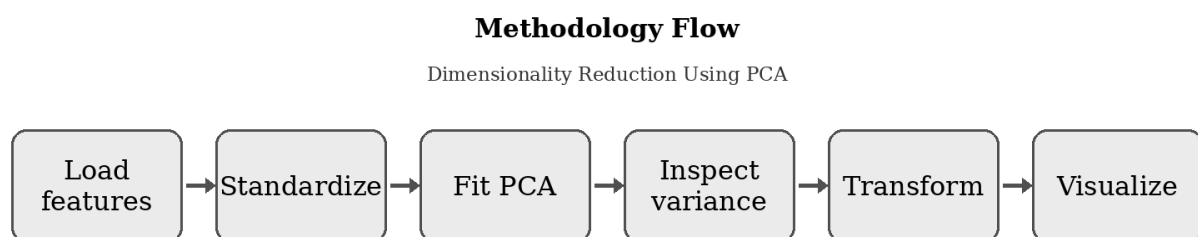


Figure 3. Methodology flow for Assignment 3.

Tools and Technologies

Tool / Technology	Role in the practical
Pandas / NumPy	Prepare numerical feature matrix
StandardScaler	Standardize input features
PCA	Compute principal components
Matplotlib	Variance and PCA plots
Scikit-learn	Implementation

Procedure

1. Load numerical features and handle missing values.
2. Exclude the target column, if present, from PCA.
3. Standardize the feature matrix.
4. Fit PCA and compute explained variance ratios.
5. Select a practical number of components from cumulative variance.
6. Transform data and visualize the first two components.

Evaluation and Expected Output

Important outputs / measures: explained variance ratio, cumulative variance, number of components, 2-D PCA scatter.

The final output should provide visible evidence that the algorithm or statistical method was executed successfully. The observation must be based on the actual values produced during execution, not on assumed results.

Result / Observation

Principal Component Analysis (PCA) was successfully applied to the **Wine dataset containing 178 observations and 13 numerical features** after standardization. The data was reduced from **13 dimensions to 2 principal components**. PC1 explained **36.20%** of the variance and PC2 explained **19.21%**, giving a total variance retention of **55.41%**. The PCA transformation provided a lower-dimensional representation of the dataset while retaining a significant portion of its original variation.

Code Attachment: assignment_03/code/assignment_03_pca.py

```
assignment_03_pca.py X ... assignment_03_pca.py X
Practical_3 > assignment_03_pca.py > ...
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 from sklearn.datasets import load_wine
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.decomposition import PCA
8
9 print("-" * 60)
10 print("ASSIGNMENT 3: DIMENSIONALITY REDUCTION USING PCA")
11 print("-" * 60)
12
13 # Load Wine dataset
14 wine = load_wine()
15
16 X = pd.DataFrame(wine.data, columns=wine.feature_names)
17 y = wine.target
18
19 print("\n--- Dataset Preview ---")
20 print(X.head())
21
22 print("\n--- Dataset Shape ---")
23 print(X.shape)
24
25 print("\n--- Number of Features ---")
26 print(X.shape[1])
27
28 print("\n--- Number of Classes ---")
29 print(len(np.unique(y)))
30
31 # Standardization
32 scaler = StandardScaler()
33 X_scaled = scaler.fit_transform(X)
34
35 print("\n--- Standardization Completed ---")
36
37 # PCA on all features to calculate explained variance
38 pca_full = PCA()
39 X_pca_full = pca_full.fit_transform(X_scaled)
40
41 explained_variance = pca_full.explained_variance_ratio_
42 cumulative_variance = np.cumsum(explained_variance)
43
44 print("\n--- Explained Variance Ratio ---")
45 for i, value in enumerate(explained_variance, start=1):
46     print(f"PC({i}): (value:.4f) ((value * 100:.2f)%)")
47
48 print("\n--- Cumulative Explained Variance ---")
49 for i, value in enumerate(cumulative_variance, start=1):
50     print(f"PC1-PC({i}): (value:.4f) ((value * 100:.2f)%)")
51
52 # PCA to 2 dimensions
53 pca_2d = PCA(n_components=2)
54 X_pca_2d = pca_2d.fit_transform(X_scaled)
55
56 print("\n--- PCA Reduction ---")
57 print(X_pca_2d.shape)
58
59 # Cumulative variance plot
60 plt.figure(figsize=(8, 5))
61 plt.plot(
62     range(1, len(cumulative_variance) + 1),
63     cumulative_variance,
64     marker="o"
65 )
66
67 plt.axhline(y=0.90, linestyle="--", label="90% variance")
68 plt.xlabel("Number of Principal Components")
69 plt.ylabel("Cumulative Explained Variance")
70 plt.title("Cumulative Explained Variance using PCA")
71 plt.grid(True)
72 plt.legend()
73 plt.tight_layout()
74 plt.savefig("cumulative_variance.png", dpi=300)
75 plt.show()
76
77 # 2-D PCA scatter plot
78 plt.figure(figsize=(8, 6))
79
80 for class_label in np.unique(y):
81     plt.scatter(
82         X_pca_2d[y == class_label, 0],
83         X_pca_2d[y == class_label, 1],
84         label=f"Class {class_label}",
85         s=60
86     )
87
88 plt.xlabel("Principal Component 1")
89 plt.ylabel("Principal Component 2")
90 plt.title("Main Dataset after PCA")
91 plt.legend()
92 plt.grid(True)
93 plt.tight_layout()
94 plt.savefig("pca_2d_scatter.png", dpi=300)
95 plt.show()
96
97 print("\n--- Execution Completed Successfully ---")
```

Output / Screenshot Attachment: assignment_03/screenshots/execution_capture.png

```
=====
ASSIGNMENT 3: DIMENSIONALITY REDUCTION USING PCA.2 -> 26.2.1
=====oft\WindowsApps\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip

--- Dataset Preview ---\Desktop\AI\ML\Practical_3>
alcohol  malic_acid  ash  alkalinity_of_ash  magnesium  ...  proanthocyanins  color_intensity  hue  od280/od315_of_diluted_wines  proline
0      14.23      1.71  2.43          15.6      127.0  ...          2.29          5.64  1.04          3.92      1065.0
1      13.20      1.78  2.14          11.2      100.0  ...          1.28          4.38  1.05          3.40      1050.0
2      13.16      2.36  2.67          18.6      101.0  ...          2.81          5.68  1.03          3.17      1185.0
3      14.37      1.95  2.50          16.8      113.0  ...          2.18          7.80  0.86          3.45      1480.0
4      13.24      2.59  2.87          21.0      118.0  ...          1.82          4.32  1.04          2.93       735.0

[5 rows x 13 columns]

--- Dataset Shape ---
(178, 13)

--- Number of Features ---
13

--- Number of Classes ---
3

--- Standardization Completed ---

--- Explained Variance Ratio ---
PC1: 0.3620 (36.20%)
PC2: 0.1921 (19.21%)
PC3: 0.1112 (11.12%)
PC4: 0.0707 (7.07%)
PC5: 0.0656 (6.56%)
PC6: 0.0494 (4.94%)
PC7: 0.0424 (4.24%)
PC8: 0.0268 (2.68%)
PC9: 0.0222 (2.22%)
PC10: 0.0193 (1.93%)
PC11: 0.0174 (1.74%)
PC12: 0.0130 (1.30%)
PC13: 0.0080 (0.80%)
```

--- Cumulative Explained Variance ---

PC1-PC1: 0.3620 (36.20%)
PC1-PC2: 0.5541 (55.41%)
PC1-PC3: 0.6653 (66.53%)
PC1-PC4: 0.7360 (73.60%)
PC1-PC5: 0.8016 (80.16%)
PC1-PC6: 0.8510 (85.10%)
PC1-PC7: 0.8934 (89.34%)
PC1-PC8: 0.9202 (92.02%)
PC1-PC9: 0.9424 (94.24%)
PC1-PC10: 0.9617 (96.17%)
PC1-PC11: 0.9791 (97.91%)
PC1-PC12: 0.9920 (99.20%)
PC1-PC13: 1.0000 (100.00%)

--- PCA Reduction ---

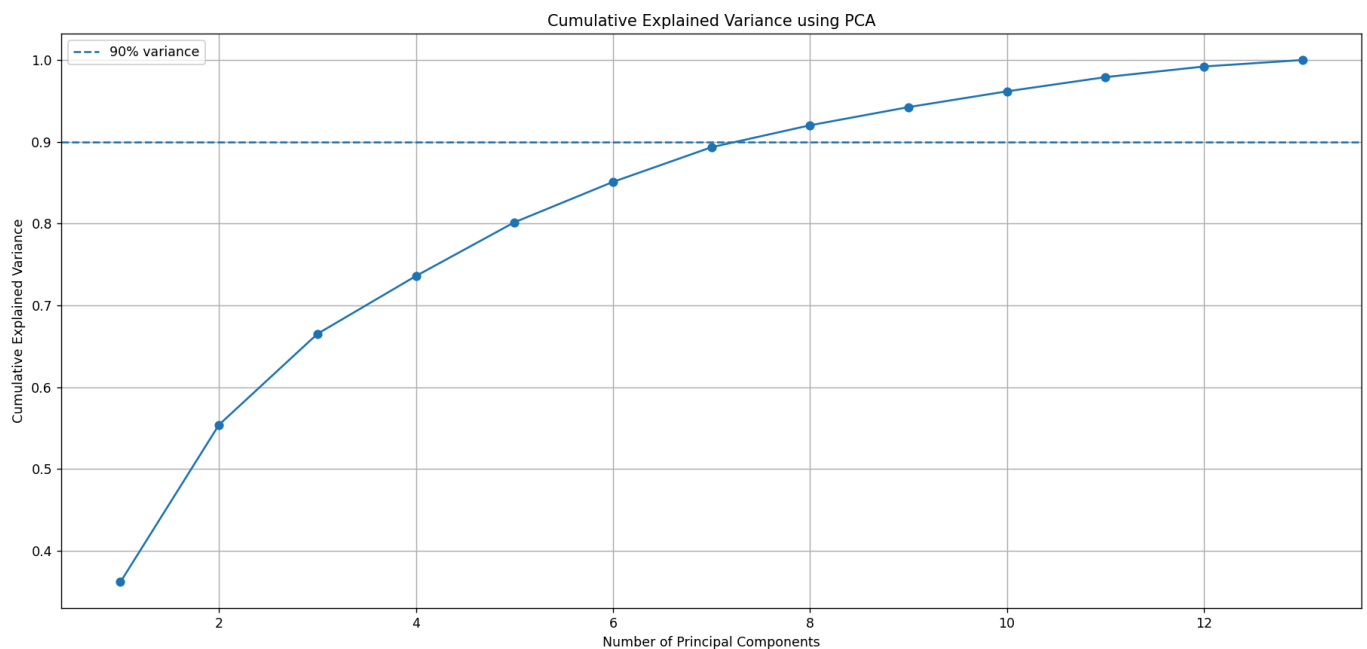
Original Dimensions : 13
Reduced Dimensions : 2

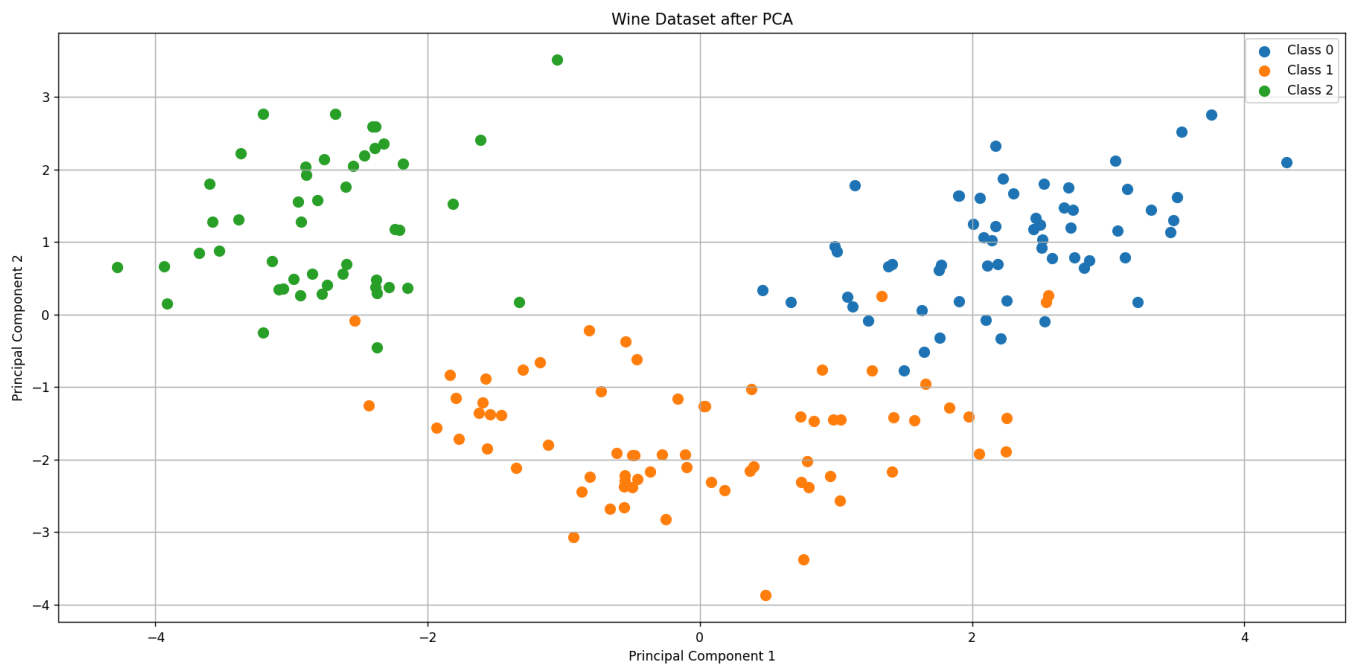
--- Variance Retained by 2 Components ---

PC1: 36.20%
PC2: 19.21%
Total: 55.41%

--- Execution Completed Successfully ---

C:\Users\hardi\OneDrive\Desktop\AIML\Practical_3>





Assignment 4: Naive Bayes Classification

Aim: Implement a Naive Bayes classifier for labeled data and evaluate its predictions with common classification measures.

Objectives

- Understand Bayes-based probabilistic classification.
- Use the conditional-independence assumption of Naive Bayes.
- Train a suitable variant such as Gaussian Naive Bayes.
- Evaluate predictions using a classification report and confusion matrix.

Key Definitions

Term	Definition
Classification	Assigning an observation to one of predefined classes.
Bayes theorem	Updates class probability using prior probability and observed evidence.
Prior	Probability of a class before the current feature evidence is considered.
Likelihood	Probability of observing the feature values given a class.
Naive Bayes	Probabilistic classifier that assumes conditional independence of features given the class.

Theory / Working Principle

- For a class C and feature vector X , Bayes theorem combines prior probability and likelihood to estimate the posterior $P(C|X)$.
- The naive conditional-independence assumption simplifies probability computation and makes the algorithm efficient.
- GaussianNB is suitable for continuous numerical features; other Naive Bayes variants are used for other feature types.

Implementation note: A small local SMS ham/spam CSV was used for text vectorization and Multinomial Naive Bayes classification.

Methodology Flow

Naive Bayes Classification



Figure 4. Methodology flow for Assignment 4.

Tools and Technologies

Tool / Technology	Role in the practical
Pandas	Dataset preparation
Scikit-learn	Naive Bayes and train/test split
Metrics	Accuracy, precision, recall, F1
Matplotlib / Seaborn	Confusion matrix
VS Code + Python	Execution

Procedure

1. Load labeled data and identify X and y.
2. Clean and encode features when needed.
3. Split data into training and testing sets.
4. Fit the Naive Bayes classifier.
5. Predict the test set and calculate metrics.
6. Display the confusion matrix and interpret errors.

Evaluation and Expected Output

Important outputs / measures: accuracy, precision, recall, F1-score, confusion matrix.

The final output should provide visible evidence that the algorithm or statistical method was executed successfully. The observation must be based on the actual values produced during execution, not on assumed results.

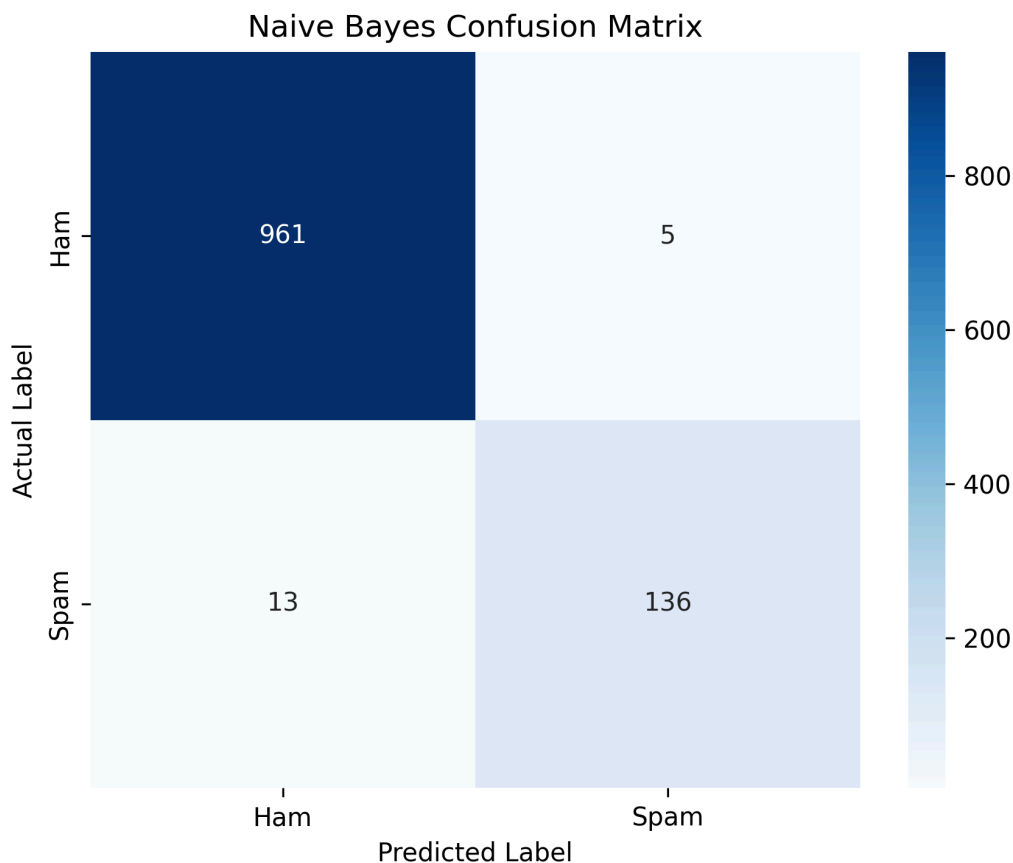
Result / Observation

The **Multinomial Naive Bayes classifier** was successfully implemented on the SMS Spam Collection dataset after converting the text messages into numerical features using Count Vectorization. The model achieved an **accuracy of 98.39%**, with **96.45% precision, 91.28% recall, and 93.79% F1-score**. The confusion matrix showed that 961 ham messages and 136 spam messages were correctly classified. The results demonstrate that Naive Bayes can effectively classify SMS messages into ham and spam categories.

Code Attachment: assignment_04/code/assignment_04_naive_bayes.py

```
Practical_4 > assignment_04_naive_bayes.py > ...
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4
5 from sklearn.model_selection import train_test_split
6 from sklearn.feature_extraction.text import CountVectorizer
7 from sklearn.naive_bayes import MultinomialNB
8 from sklearn.metrics import (
9     accuracy_score,
10     precision_score,
11     recall_score,
12     f1_score,
13     confusion_matrix,
14     classification_report
15 )
16
17 print("-" * 60)
18 print("ASSIGNMENT 4: NAIVE BAYES CLASSIFICATION")
19 print("-" * 60)
20
21 # Load dataset
22 df = pd.read_csv("spam.csv", encoding="latin-1")
23
24 # Keep only the required columns
25 df = df.iloc[:, :2]
26 df.columns = ["label", "message"]
27
28 print("\n--- Dataset Preview ---")
29 print(df.head())
30
31 print("\n--- Dataset Shape ---")
32 print(df.shape)
33
34 print("\n--- Class Distribution ---")
35 print(df["label"].value_counts())
36
37 # Convert labels to numerical values
38 df["label_num"] = df["label"].map({"ham": 0, "spam": 1})
39
40 X = df["message"]
41 y = df["label_num"]
42
43 # Split data
44 X_train, X_test, y_train, y_test = train_test_split(
45     X,
46     y,
47     test_size=0.20,
48     random_state=42,
49     stratify=y
50 )
51
52 print("\n--- Train/Test Split ---")
53 print(f"Training samples : {len(X_train)}")
54 print(f"Testing samples : {len(X_test)}")
55
56 # Convert text to numerical features
57 vectorizer = CountVectorizer()
58 X_train_vec = vectorizer.fit_transform(X_train)
59 X_test_vec = vectorizer.transform(X_test)
60
61 print("\n--- Text Vectorization ---")
62 print(f"Training matrix shape : {X_train_vec.shape}")
63 print(f"Testing matrix shape : {X_test_vec.shape}")
64
65 # Train Multinomial Naive Bayes
66 model = MultinomialNB()
67 model.fit(X_train_vec, y_train)
68
69 # Predictions
70 y_pred = model.predict(X_test_vec)
71
72 # Metrics
73 accuracy = accuracy_score(y_test, y_pred)
74 precision = precision_score(y_test, y_pred)
75 recall = recall_score(y_test, y_pred)
76 f1 = f1_score(y_test, y_pred)
77
78 print("\n--- Naive Bayes Results ---")
79 print(f"Accuracy : {accuracy:.4f}")
80 print(f"Precision : {precision:.4f}")
81 print(f"Recall : {recall:.4f}")
82 print(f"F1-Score : {f1:.4f}")
83
84 print("\n--- Classification Report ---")
85 print(
86     classification_report(
87         y_test,
88         y_pred,
89         target_names=["Ham", "Spam"]
90     )
91 )
92
93 # Confusion matrix
94 cm = confusion_matrix(y_test, y_pred)
95
96 print("\n--- Confusion Matrix ---")
97 print(cm)
98
99 plt.figure(figsize=(6, 5))
100 sns.heatmap(
101     cm,
102     annot=True,
103     fmt="d",
104     cmap="Blues",
105     xticklabels=["Ham", "Spam"],
106     yticklabels=["Ham", "Spam"]
107 )
108
109 plt.xlabel("Predicted Label")
110 plt.ylabel("Actual Label")
111 plt.title("Naive Bayes Confusion Matrix")
112 plt.tight_layout()
113 plt.savefig("confusion_matrix.png", dpi=300)
```

Output / Screenshot Attachment: assignment_04/screenshots/execution_capture.png



=====

ASSIGNMENT 4: NAIVE BAYES CLASSIFICATION

=====

--- Dataset Preview ---

	label	message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

--- Dataset Shape ---

(5572, 2)

--- Class Distribution ---

label	
ham	4825
spam	747

Name: count, dtype: int64

--- Train/Test Split ---

Training samples : 4457
Testing samples : 1115

--- Text Vectorization ---

Training matrix shape : (4457, 7701)
Testing matrix shape : (1115, 7701)

--- Naive Bayes Results ---

Accuracy : 0.9839
Precision : 0.9645
Recall : 0.9128
F1-Score : 0.9379

--- Classification Report ---

	precision	recall	f1-score	support
Ham	0.99	0.99	0.99	966
Spam	0.96	0.91	0.94	149
accuracy			0.98	1115
macro avg	0.98	0.95	0.96	1115
weighted avg	0.98	0.98	0.98	1115

--- Confusion Matrix ---

[[961 5]
[13 136]]

Assignment 5: KNN Classification

Aim: Implement the K-Nearest Neighbours algorithm for classification and observe how the chosen neighbourhood size affects performance.

Objectives

- Understand instance-based classification.
- Apply scaling before distance-based learning.
- Compare several practical K values.
- Evaluate the final classifier with a confusion matrix and classification metrics.

Key Definitions

Term	Definition
KNN	Predicts a class from the labels of the K closest training observations.
Euclidean distance	Common distance measure based on squared coordinate differences.
K value	Number of neighbouring observations considered for a prediction.
Feature scaling	Transforms features to comparable scales so distance is meaningful.
Lazy learner	Stores training examples and performs most prediction work at query time.

Theory / Working Principle

- KNN classifies an observation according to the classes of nearby training examples, often using majority voting.
- Small K values can be sensitive to noise; larger K values create smoother decision boundaries.
- Scaling is important because the algorithm relies on distances between feature vectors.

Implementation note: The scikit-learn built-in Iris dataset was used for KNN classification.

Methodology Flow

KNN Classification

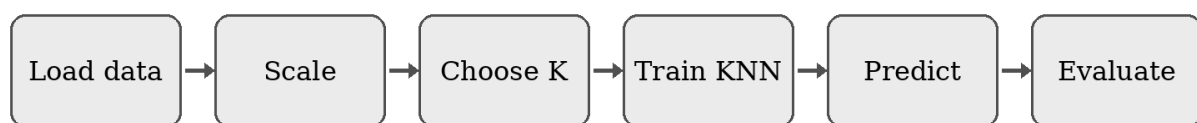


Figure 5. Methodology flow for Assignment 5.

Tools and Technologies

Tool / Technology	Role in the practical
Pandas / NumPy	Data preparation
StandardScaler	Feature scaling
KNeighborsClassifier	KNN model
Scikit-learn metrics	Evaluation
Matplotlib / Seaborn	Plots and confusion matrix

Procedure

1. Load a labeled classification dataset.
2. Separate features and target labels.
3. Split into training and testing data.
4. Fit the scaler on training data and transform both sets.
5. Evaluate several K values and select a practical value.
6. Predict, calculate metrics and inspect the confusion matrix.

Evaluation and Expected Output

Important outputs / measures: accuracy, precision, recall, F1-score, confusion matrix.

The final output should provide visible evidence that the algorithm or statistical method was executed successfully. The observation must be based on the actual values produced during execution, not on assumed results.

Result / Observation

The K-Nearest Neighbors (KNN) classifier was successfully implemented on the Iris dataset after standardizing the input features. Different values of K were compared, and K = 1 produced the highest test accuracy of 96.67%. The final model achieved 96.97% precision, 96.67% recall, and 96.66% F1-score. The confusion matrix showed that all Setosa and Versicolor samples were correctly classified, while 9 out of 10 Virginica samples were correctly classified.

Code Attachment: assignment_05/code/assignment_05_knn.py

```
assignment_05_knn.py
Practical_5 > assignment_05_knn.py > ...
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4
5 from sklearn.datasets import load_iris
6 from sklearn.model_selection import train_test_split
7 from sklearn.preprocessing import StandardScaler
8 from sklearn.neighbors import KNeighborsClassifier
9 from sklearn.metrics import (
10     accuracy_score,
11     precision_score,
12     recall_score,
13     f1_score,
14     confusion_matrix,
15     classification_report
16 )
17
18 print("=" * 60)
19 print("ASSIGNMENT 5: KNN CLASSIFICATION")
20 print("=" * 60)
21
22 # Load Iris dataset
23 iris = load_iris()
24 X = pd.DataFrame(iris.data, columns=iris.feature_names)
25 y = iris.target
26
27 print("\n--- Dataset Preview ---")
28 print(X.head())
29
30 print("\n--- Dataset Shape ---")
31 print(X.shape)
32
33 print("\n--- Number of Classes ---")
34 print(len(iris.target_names))
35
36 # Train-test split
37 X_train, X_test, y_train, y_test = train_test_split(
38     X,
39     y,
40     test_size=0.20,
41     random_state=42,
42     stratify=y
43 )
44
45 print("\n--- Train/Test Split ---")
46 print(f"Training samples : {len(X_train)}")
47 print(f"Testing samples : {len(X_test)}")
48
49 # Feature scaling
50 scaler = StandardScaler()
51 X_train_scaled = scaler.fit_transform(X_train)
52 X_test_scaled = scaler.transform(X_test)
53
54 print("\n--- Feature Scaling ---")
55 print("StandardScaler applied successfully.")
56
57 # Compare different K values
58 k_values = [1, 3, 5, 7, 9]
59 accuracies = []
60
61 print("\n--- K Value Comparison ---")
62
63 for k in k_values:
64     knn = KNeighborsClassifier(n_neighbors=k)
65     knn.fit(X_train_scaled, y_train)
66
67     predictions = knn.predict(X_test_scaled)
68     accuracy = accuracy_score(y_test, predictions)
69     accuracies.append(accuracy)
70
71     print(f"K={k}: Accuracy={accuracy:.4f}")
72
73 # Select best K based on highest test accuracy
74 best_index = accuracies.index(max(accuracies))
75 best_k = k_values[best_index]
76
77 print("\n--- Selected K ---")
78 print(f"Best K : {best_k}")
79 print(f"Best Accuracy : {accuracies[best_index]:.4f}")
80
81 # Final model
82 final_knn = KNeighborsClassifier(n_neighbors=best_k)
83 final_knn.fit(X_train_scaled, y_train)
84
85
86 y_pred = final_knn.predict(X_test_scaled)
87
88 # Final metrics
89 accuracy = accuracy_score(y_test, y_pred)
90 precision = precision_score(y_test, y_pred, average="weighted")
91 recall = recall_score(y_test, y_pred, average="weighted")
92 f1 = f1_score(y_test, y_pred, average="weighted")
93
94 print("\n--- Final KNN Results ---")
95 print(f"Accuracy : {accuracy:.4f}")
96 print(f"Precision : {precision:.4f}")
97 print(f"Recall : {recall:.4f}")
98 print(f"F1-Score : {f1:.4f}")
99
100 print("\n--- Classification Report ---")
101 print(
102     classification_report(
103         y_test,
104         y_pred,
105         target_names=iris.target_names
106     )
107 )
108
109 # Confusion matrix
110 cm = confusion_matrix(y_test, y_pred)
111
112 print("\n--- Confusion Matrix ---")
113 print(cm)
114
115 plt.figure(figsize=(6, 5))
116
117 sns.heatmap(
118     cm,
119     annot=True,
120     fmt="d",
121     cmap="Blues",
122     xticklabels=iris.target_names,
123     yticklabels=iris.target_names
124 )
125
126 plt.xlabel("Predicted Label")
127 plt.ylabel("Actual Label")
128 plt.title(f"KNN Confusion Matrix (K={best_k})")
129 plt.tight_layout()
130 plt.savefig("knn_confusion_matrix.png", dpi=300)
131 plt.show()
132
133 # Accuracy comparison plot
134 plt.figure(figsize=(8, 5))
135
136 plt.plot(
137     k_values,
138     accuracies,
139     marker="o"
140 )
141
142 plt.xlabel("K Value")
143 plt.ylabel("Accuracy")
144 plt.title("KNN Accuracy for Different K Values")
145 plt.xticks(k_values)
146 plt.grid(True)
147 plt.tight_layout()
148 plt.savefig("knn_k_comparison.png", dpi=300)
149 plt.show()
150
151 print("\n--- Execution Completed Successfully ---")
```

```
assignment_05_knn.py
Practical_5 > assignment_05_knn.py > ...
86 y_pred = final_knn.predict(X_test_scaled)
87
88 # Final metrics
89 accuracy = accuracy_score(y_test, y_pred)
90 precision = precision_score(y_test, y_pred, average="weighted")
91 recall = recall_score(y_test, y_pred, average="weighted")
92 f1 = f1_score(y_test, y_pred, average="weighted")
93
94 print("\n--- Final KNN Results ---")
95 print(f"Accuracy : {accuracy:.4f}")
96 print(f"Precision : {precision:.4f}")
97 print(f"Recall : {recall:.4f}")
98 print(f"F1-Score : {f1:.4f}")
99
100 print("\n--- Classification Report ---")
101 print(
102     classification_report(
103         y_test,
104         y_pred,
105         target_names=iris.target_names
106     )
107 )
108
109 # Confusion matrix
110 cm = confusion_matrix(y_test, y_pred)
111
112 print("\n--- Confusion Matrix ---")
113 print(cm)
114
115 plt.figure(figsize=(6, 5))
116
117 sns.heatmap(
118     cm,
119     annot=True,
120     fmt="d",
121     cmap="Blues",
122     xticklabels=iris.target_names,
123     yticklabels=iris.target_names
124 )
125
126 plt.xlabel("Predicted Label")
127 plt.ylabel("Actual Label")
128 plt.title(f"KNN Confusion Matrix (K={best_k})")
129 plt.tight_layout()
130 plt.savefig("knn_confusion_matrix.png", dpi=300)
131 plt.show()
132
133 # Accuracy comparison plot
134 plt.figure(figsize=(8, 5))
135
136 plt.plot(
137     k_values,
138     accuracies,
139     marker="o"
140 )
141
142 plt.xlabel("K Value")
143 plt.ylabel("Accuracy")
144 plt.title("KNN Accuracy for Different K Values")
145 plt.xticks(k_values)
146 plt.grid(True)
147 plt.tight_layout()
148 plt.savefig("knn_k_comparison.png", dpi=300)
149 plt.show()
150
151 print("\n--- Execution Completed Successfully ---")
```

Output / Screenshot Attachment: assignment_05/screenshots/execution_capture.png

=====

ASSIGNMENT 5: KNN CLASSIFICATION

=====

--- Dataset Preview ---

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

--- Dataset Shape ---

(150, 4)

--- Number of Classes ---

3

--- Train/Test Split ---

Training samples : 120

Testing samples : 30

--- Feature Scaling ---

StandardScaler applied successfully.

--- K Value Comparison ---

K=1: Accuracy=0.9667

K=3: Accuracy=0.9333

K=5: Accuracy=0.9333

K=7: Accuracy=0.9667

K=9: Accuracy=0.9667

--- Selected K ---

Best K : 1

Best Accuracy : 0.9667

--- Final KNN Results ---

Accuracy : 0.9667

Precision : 0.9697

Recall : 0.9667

F1-Score : 0.9666

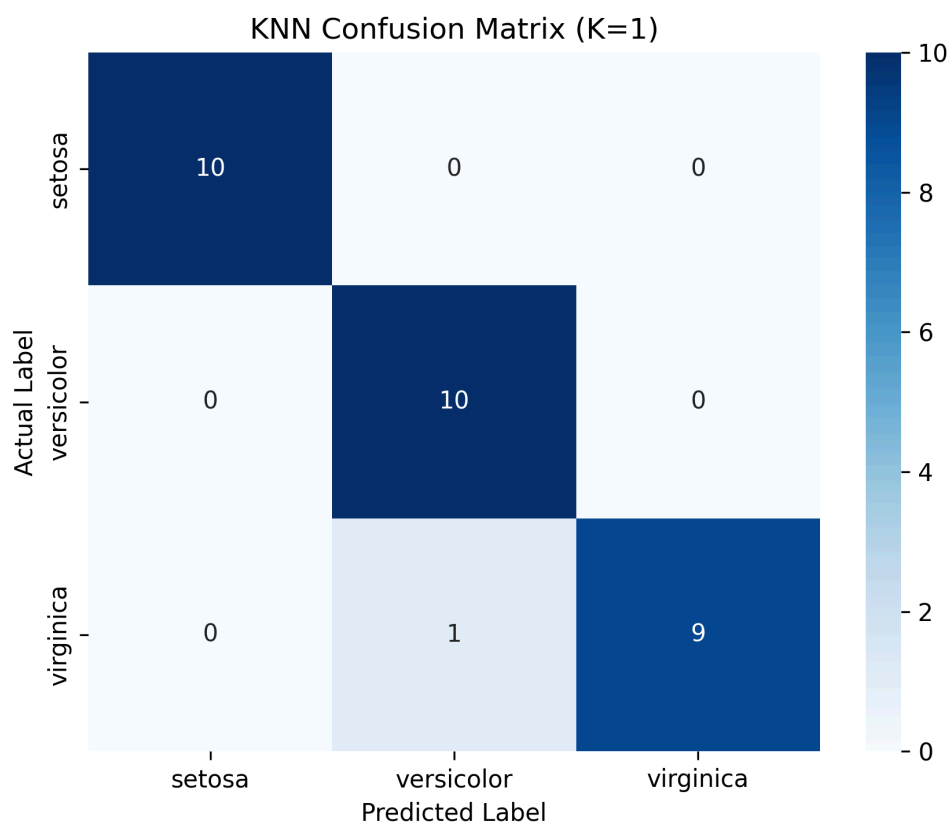
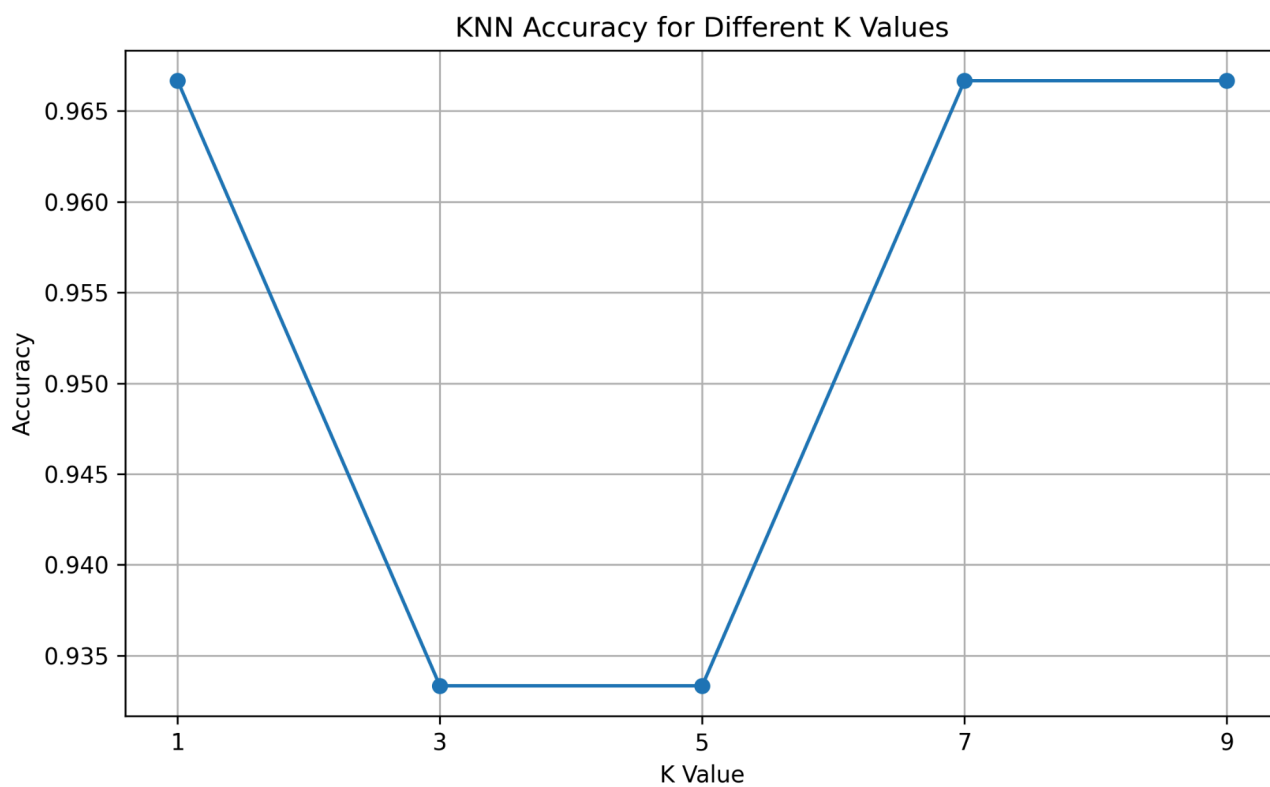
--- Classification Report ---

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.91	1.00	0.95	10
virginica	1.00	0.90	0.95	10
accuracy			0.97	30
macro avg	0.97	0.97	0.97	30
weighted avg	0.97	0.97	0.97	30

--- Confusion Matrix ---

```
[[10  0  0]
 [ 0 10  0]
 [ 0  1  9]]
```

--- Execution Completed Successfully ---



Assignment 6: Linear Regression

Aim: Implement Linear Regression to estimate a continuous dependent variable from one or more independent variables and evaluate the prediction error.

Objectives

- Understand regression as prediction of a continuous target.
- Fit a least-squares linear model.
- Interpret coefficients and intercept.
- Evaluate the model on unseen test data.

Key Definitions

Term	Definition
Regression	Supervised learning for a continuous numerical target.
Linear regression	Represents the expected target as a linear combination of predictors.
Coefficient	Estimated change in predicted target per unit change in a predictor, holding other predictors fixed in multiple regression.
Intercept	Predicted target when all predictors are zero in the fitted equation.
Residual	Observed value minus predicted value.

Theory / Working Principle

- Ordinary least squares estimates coefficients by minimizing the sum of squared residuals.
- For one predictor, the model is a straight line; with several predictors, it is a hyperplane.
- A separate test set provides a basic check of generalization to unseen observations.

Implementation note: A small local house-price CSV containing area and price was used for simple linear regression.

Methodology Flow

Linear Regression

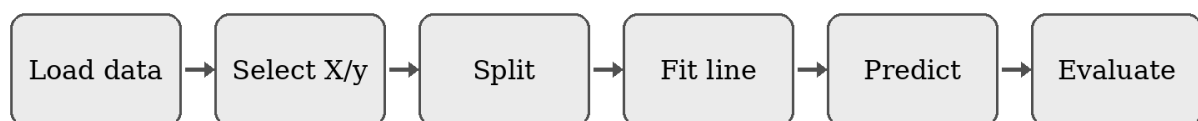


Figure 6. Methodology flow for Assignment 6.

Tools and Technologies

Tool / Technology	Role in the practical
Pandas / NumPy	Data preparation
LinearRegression	Fit model
train_test_split	Create train/test sets
Metrics	MAE, MSE, RMSE, R2
Matplotlib	Regression/prediction plot

Procedure

1. Load and inspect numerical data.
2. Choose predictor columns X and continuous target y.
3. Handle missing values.
4. Split into training and testing sets.
5. Fit LinearRegression on the training set.
6. Predict the test set, calculate metrics and plot results.

Evaluation and Expected Output

Important outputs / measures: MAE, MSE, RMSE, R2 score, actual vs predicted plot.

The final output should provide visible evidence that the algorithm or statistical method was executed successfully. The observation must be based on the actual values produced during execution, not on assumed results.

Result / Observation

Linear Regression was successfully implemented on the **Diabetes dataset** using **BMI** as the independent variable and the disease progression **target** as the dependent variable. The dataset was divided into **353 training samples and 89 testing samples**. The model obtained an **MAE of 52.2600, MSE of 4061.8259, RMSE of 63.7325, and R² of 0.2334** on the test data. The actual-versus-predicted plot was used to visualize the model predictions and regression performance.

Code Attachment: assignment_06/code/assignment_06_linear_regression.py

```
assignment_05_tnn.py × assignment_06_linear_regression.py × ... assignment_06_linear_regression.py ×
Practical_6 > assignment_06_linear_regression.py > ...
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 from sklearn.datasets import load_diabetes
5 from sklearn.model_selection import train_test_split
6 from sklearn.linear_model import LinearRegression
7 from sklearn.metrics import (
8     mean_absolute_error,
9     mean_squared_error,
10     r2_score
11 )
12
13 print("=" * 60)
14 print("ASSIGNMENT 6: LINEAR REGRESSION")
15 print("=" * 60)
16
17 # Load dataset
18 diabetes = load_diabetes(as_frame=True)
19
20 df = diabetes.frame
21
22 print("\n--- Dataset Preview ---")
23 print(df.head())
24
25 print("\n--- Dataset Shape ---")
26 print(df.shape)
27
28 print("\n--- Dataset Columns ---")
29 print(df.columns.tolist())
30
31 # Select independent and dependent variables
32 X = df[["bmi"]]
33 y = df["target"]
34
35 print("\n--- Variables ---")
36 print("Independent Variable : bmi")
37 print("Dependent Variable : target")
38
39 # Split data
40 X_train, X_test, y_train, y_test = train_test_split(
41     X,
42     y,
43     test_size=0.20,
44     random_state=42
45 )
46
47 print("\n--- Train/Test Split ---")
48 print(f"Training samples : {len(X_train)}")
49 print(f"Testing samples : {len(X_test)}")
50
51
52 # Model parameters
53 print("\n--- Linear Regression Model ---")
54 print(f"Intercept : {model.intercept_:.4f}")
55 print(f"Coefficient : {model.coef_[0]:.4f}")
56
57 # Predictions
58 y_pred = model.predict(X_test)
59
60 # Metrics
61 mae = mean_absolute_error(y_test, y_pred)
62 mse = mean_squared_error(y_test, y_pred)
63 rmse = mse ** 0.5
64 r2 = r2_score(y_test, y_pred)
65
66 print("\n--- Regression Results ---")
67 print(f"MAE : {mae:.4f}")
68 print(f"MSE : {mse:.4f}")
69 print(f"RMSE : {rmse:.4f}")
70 print(f"R^2 : {r2:.4f}")
71
72 # Actual vs predicted plot
73 plt.figure(figsize=(8, 6))
74
75 plt.scatter(
76     y_test,
77     y_pred,
78     s=60,
79     alpha=0.7
80 )
81
82 plt.plot(
83     [y_test.min(), y_test.max()],
84     [y_test.min(), y_test.max()],
85     linestyle="--"
86 )
87
88 plt.xlabel("Actual Target")
89 plt.ylabel("Predicted Target")
90 plt.title("Actual vs Predicted Values - Linear Regression")
91
92 plt.grid(True)
93
94 plt.tight_layout()
95
96 plt.savefig("actual_vs_predicted.png", dpi=300)
97 plt.show()
98
99 print("\n--- Sample Predictions ---")
100
101 comparison = pd.DataFrame({
102     "Actual": y_test.values[:10],
103     "Predicted": y_pred[:10]
104 })
```

Output / Screenshot Attachment: assignment_06/screenshots/execution_capture.png

```
=====
ASSIGNMENT 6: LINEAR REGRESSION
=====

--- Dataset Preview ---
   age    sex    bmi    bp    s1    s2    s3    s4    s5    s6  target
0  0.038076  0.050680  0.061696  0.021872 -0.044223 -0.034821 -0.043401 -0.002592  0.019907 -0.017646  151.0
1 -0.001882 -0.044642 -0.051474 -0.026328 -0.008449 -0.019163  0.074412 -0.039493 -0.068332 -0.092204   75.0
2  0.085299  0.050680  0.044451 -0.005670 -0.045599 -0.034194 -0.032356 -0.002592  0.002861 -0.025930  141.0
3 -0.089063 -0.044642 -0.011595 -0.036656  0.012191  0.024991 -0.036038  0.034309  0.022688 -0.009362  206.0
4  0.005383 -0.044642 -0.036385  0.021872  0.003935  0.015596  0.008142 -0.002592 -0.031988 -0.046641  135.0

--- Dataset Shape ---
(442, 11)

--- Dataset Columns ---
['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6', 'target']

--- Variables ---
Independent Variable : bmi
Dependent Variable : target

--- Train/Test Split ---
Training samples : 353
Testing samples : 89

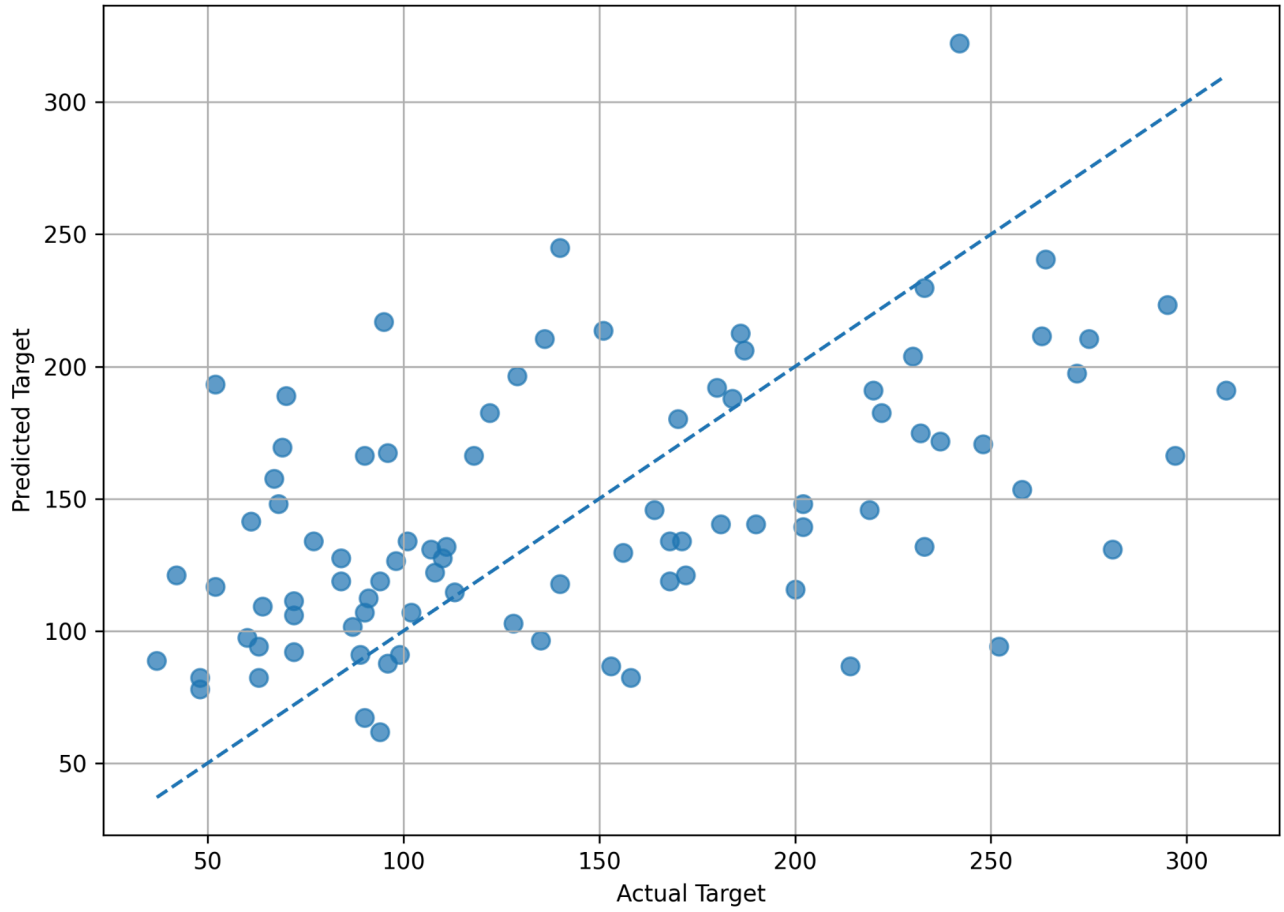
--- Linear Regression Model ---
Intercept : 152.0034
Coefficient : 998.5777

--- Regression Results ---
MAE : 52.2600
MSE : 4061.8259
RMSE : 63.7325
R : 0.2334

--- Sample Predictions ---
Actual Predicted
0 219.0 145.81
1 70.0 188.86
2 202.0 147.96
3 230.0 203.93
4 111.0 131.81
5 84.0 127.51
6 242.0 322.32
7 272.0 197.47
8 94.0 61.86
9 96.0 167.33

--- Execution Completed Successfully ---
```

Actual vs Predicted Values - Linear Regression



Assignment 7: SVM Algorithm for Regression

Aim: Implement Support Vector Regression (SVR) to estimate a continuous target using a kernel-based model with controlled error tolerance.

Objectives

- Understand the regression form of Support Vector Machines.
- Scale features before kernel-based SVR.
- Recognize the roles of C, epsilon and kernel choice.
- Evaluate predictions using regression metrics.

Key Definitions

Term	Definition
SVR	SVM-based regression that fits a function while controlling errors outside an epsilon-insensitive region.
Kernel	Similarity function that enables modeling in a transformed feature space.
Epsilon	Width of the error-insensitive region around the prediction function.
C	Regularization parameter controlling the trade-off between model complexity and training errors.
Support vector	Training point that influences the final regression function.

Theory / Working Principle

- SVR attempts to keep predictions inside an epsilon tube around the target values while controlling model complexity.
- RBF kernels can model nonlinear relationships, while linear kernels give a simpler model.
- Feature scaling is important because kernel computations depend on feature magnitude.

Implementation note: A small local advertising-index and sales dataset was used to demonstrate nonlinear SVR regression.

Methodology Flow

SVM Algorithm for Regression



Figure 7. Methodology flow for Assignment 7.

Tools and Technologies

Tool / Technology	Role in the practical
Pandas / NumPy	Input preparation
StandardScaler	Feature scaling
SVR	Regression model
Pipeline	Consistent preprocessing and model fitting
Matplotlib	Actual vs predicted plot

Procedure

1. Load and inspect a manageable regression dataset.
2. Separate features X and target y.
3. Split the data and build a scaling + SVR pipeline.
4. Start with simple kernel, C and epsilon values.
5. Predict the test observations and calculate metrics.
6. Plot actual versus predicted values and record the chosen settings.

Evaluation and Expected Output

Important outputs / measures: MAE, MSE, RMSE, R2 score, kernel / C / epsilon.

The final output should provide visible evidence that the algorithm or statistical method was executed successfully. The observation must be based on the actual values produced during execution, not on assumed results.

Result / Observation

Support Vector Regression (SVR) was successfully implemented on the Diabetes dataset using BMI as the predictor and the disease progression target as the dependent variable. An RBF kernel was used with epsilon = 0.1, and different values of C were compared. The highest observed R^2 value was 0.2086 at $C = 10$. The final SVR model achieved an MAE of 51.1714, MSE of 4193.0734, RMSE of 64.7539, and R^2 of 0.2086 on the test data.

Code Attachment: assignment_07/code/assignment_07_svr.py

```
Practical_7 > assignment_07_svr.py > ...
44
45 print("\n--- Train/Test Split ---")
46 print(f"Training samples : {len(X_train)}")
47 print(f"Testing samples : {len(X_test)}")
48
49 # Compare C values
50 c_values = [0.1, 1, 10, 100]
51 svr_scores = []
52
53 print("\n--- C Parameter Comparison ---")
54
55 for c in c_values:
56     model = Pipeline([
57         ("scaler", StandardScaler()),
58         ("svr", SVR(kernel="rbf", C=c, epsilon=0.1))
59     ])
60
61     model.fit(X_train, y_train)
62     predictions = model.predict(X_test)
63
64     rmse = mean_squared_error(y_test, predictions) ** 0.5
65     r2 = r2_score(y_test, predictions)
66
67     svr_scores.append(r2)
68
69     print(f"C={c}: RMSE={rmse:.4f}, R²={r2:.4f}")
70
71 # Select C with highest R²
72 best_index = svr_scores.index(max(svr_scores))
73 best_c = c_values[best_index]
74
75 print("\n--- Selected SVR Parameters ---")
76 print(f"Kernel : RBF")
77 print(f"C : {best_c}")
78 print(f"Epsilon : 0.1")
79
80 # Final SVR model
81 final_model = Pipeline([
82     ("scaler", StandardScaler()),
83     ("svr", SVR(
84         kernel="rbf",
85         C=best_c,
86         epsilon=0.1
```

```
Practical_7 > assignment_07_svr.py > ...
44
45 print("\n--- Train/Test Split ---")
46 print(f"Training samples : {len(X_train)}")
47 print(f"Testing samples : {len(X_test)}")
48
49 # Compare C values
50 c_values = [0.1, 1, 10, 100]
51 svr_scores = []
52
53 print("\n--- C Parameter Comparison ---")
54
55 for c in c_values:
56     model = Pipeline([
57         ("scaler", StandardScaler()),
58         ("svr", SVR(kernel="rbf", C=c, epsilon=0.1))
59     ])
60
61     model.fit(X_train, y_train)
62     predictions = model.predict(X_test)
63
64     rmse = mean_squared_error(y_test, predictions) ** 0.5
65     r2 = r2_score(y_test, predictions)
66
67     svr_scores.append(r2)
68
69     print(f"C={c}: RMSE={rmse:.4f}, R²={r2:.4f}")
70
71 # Select C with highest R²
72 best_index = svr_scores.index(max(svr_scores))
73 best_c = c_values[best_index]
74
75 print("\n--- Selected SVR Parameters ---")
76 print(f"Kernel : RBF")
77 print(f"C : {best_c}")
78 print(f"Epsilon : 0.1")
79
80 # Final SVR model
81 final_model = Pipeline([
82     ("scaler", StandardScaler()),
83     ("svr", SVR(
84         kernel="rbf",
85         C=best_c,
86         epsilon=0.1
```

```

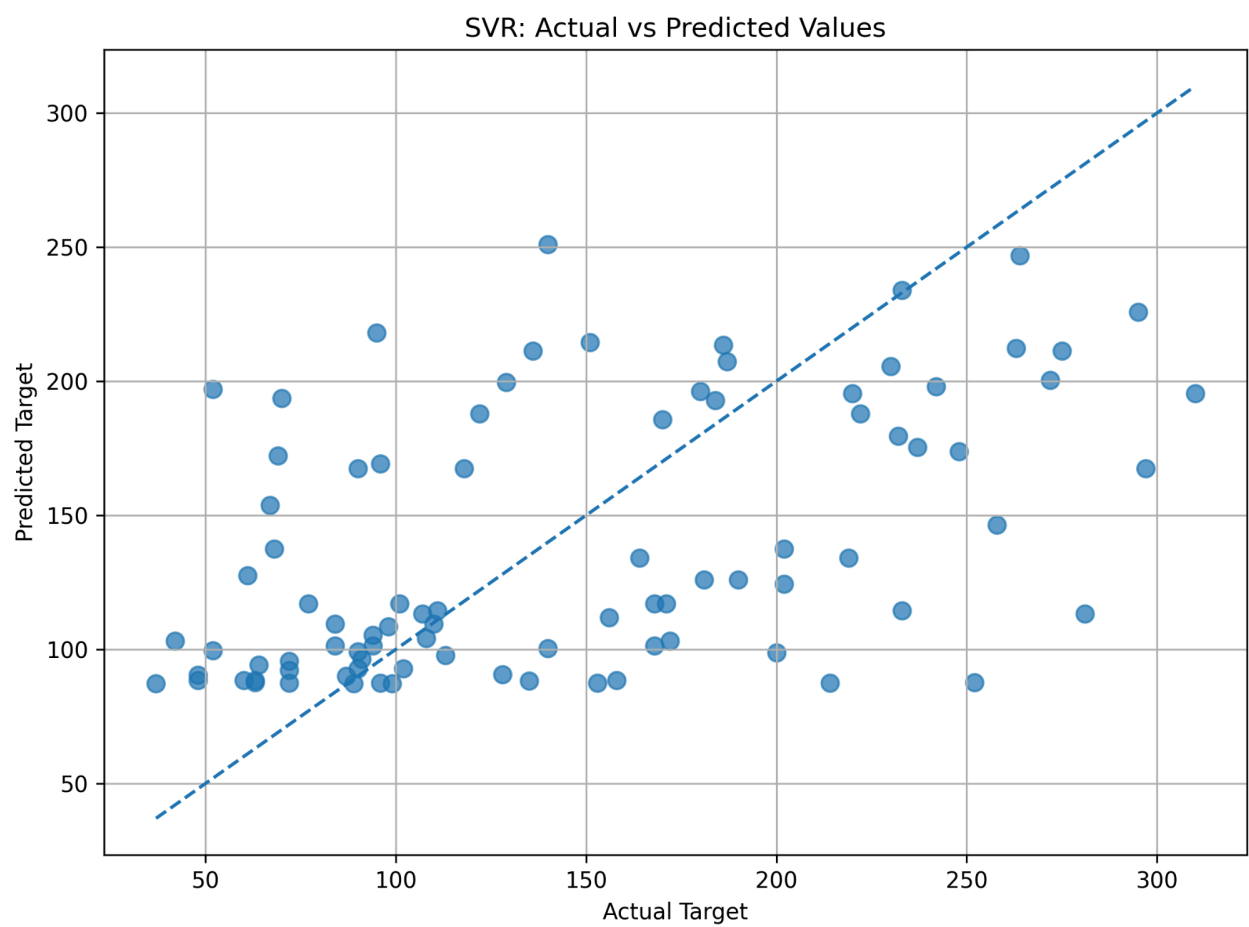
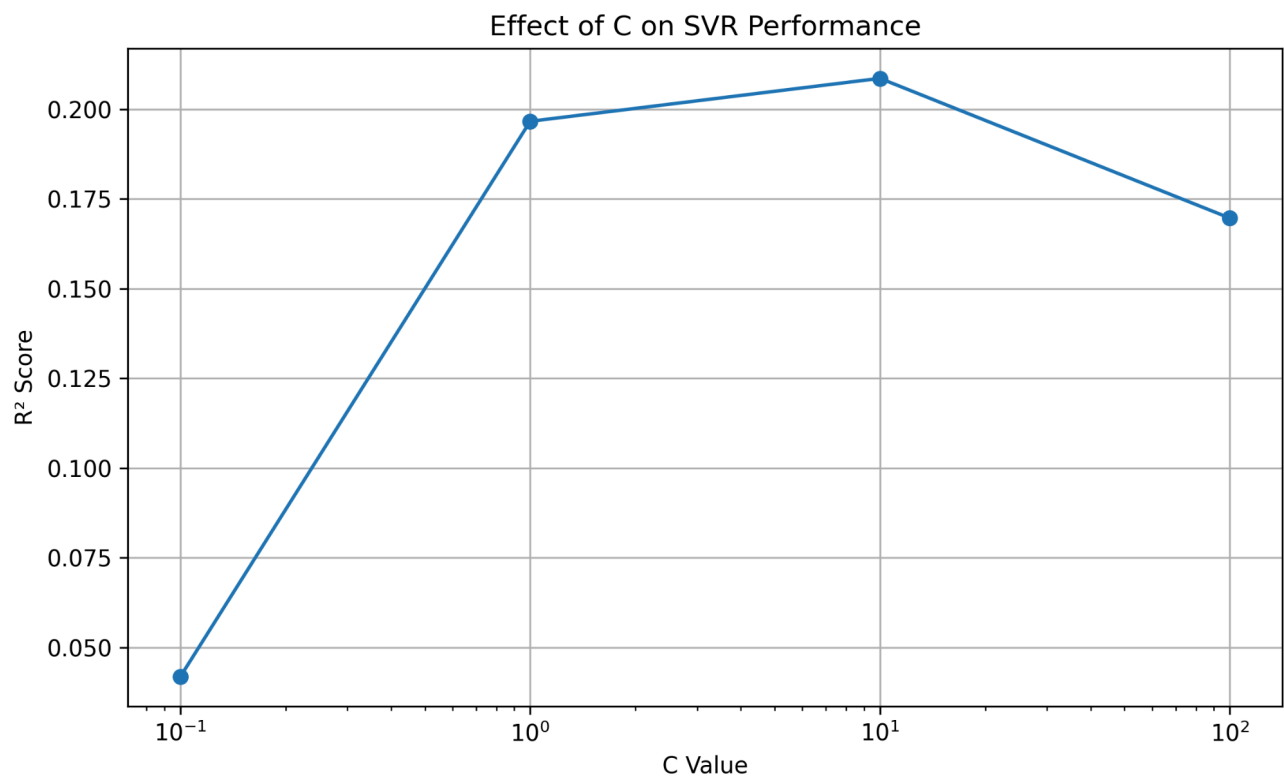
Practical_7 > assignment_07_svr.py > ...
86     epsilon=0.1
87     ))
88 ])
89
90 final_model.fit(X_train, y_train)
91
92 # Predictions
93 y_pred = final_model.predict(X_test)
94
95 # Metrics
96 mae = mean_absolute_error(y_test, y_pred)
97 mse = mean_squared_error(y_test, y_pred)
98 rmse = mse ** 0.5
99 r2 = r2_score(y_test, y_pred)
100
101 print("\n--- Final SVR Results ---")
102 print(f"MAE : {mae:.4f}")
103 print(f"MSE : {mse:.4f}")
104 print(f"RMSE : {rmse:.4f}")
105 print(f"R² : {r2:.4f}")
106
107 # Actual vs predicted
108 plt.figure(figsize=(8, 6))
109
110 plt.scatter(
111     y_test,
112     y_pred,
113     s=60,
114     alpha=0.7
115 )
116
117 plt.plot(
118     [y_test.min(), y_test.max()],
119     [y_test.min(), y_test.max()],
120     linestyle="--"
121 )
122
123 plt.xlabel("Actual Target")
124 plt.ylabel("Predicted Target")
125 plt.title("SVR: Actual vs Predicted Values")
126 plt.grid(True)
127 plt.tight_layout()

```

```

Practical_7 > assignment_07_svr.py > ...
123 plt.xlabel("Actual Target")
124 plt.ylabel("Predicted Target")
125 plt.title("SVR: Actual vs Predicted Values")
126 plt.grid(True)
127 plt.tight_layout()
128
129 plt.savefig("svr_actual_vs_predicted.png", dpi=300)
130 plt.show()
131
132 # C comparison graph
133 plt.figure(figsize=(8, 5))
134
135 plt.plot(
136     c_values,
137     svr_scores,
138     marker="o"
139 )
140
141 plt.xscale("log")
142 plt.xlabel("C Value")
143 plt.ylabel("R² Score")
144 plt.title("Effect of C on SVR Performance")
145 plt.grid(True)
146 plt.tight_layout()
147
148 plt.savefig("svr_c_comparison.png", dpi=300)
149 plt.show()
150
151 print("\n--- Execution Completed Successfully ---")

```



Assignment 8: Regularization to Avoid Overfitting

Aim: Apply Ridge and Lasso regularization to control model complexity, compare them with an unregularized linear model, and study the effect on generalization.

Objectives

- Understand overfitting and underfitting.
- Relate regularization to the bias-variance trade-off.
- Compare Linear Regression with Ridge and Lasso.
- Observe coefficient shrinkage and test-set performance.

Key Definitions

Term	Definition
Overfitting	Model fits training-specific patterns and generalizes less effectively to unseen data.
Regularization	Adds a complexity penalty to discourage excessively large coefficients.
Ridge	Linear regression with an L2 penalty that shrinks coefficients toward zero.
Lasso	Linear regression with an L1 penalty that can shrink some coefficients to exactly zero.
Bias-variance trade-off	Balance between excessive simplicity and excessive sensitivity to the training sample.

Theory / Working Principle

- Ridge penalizes the squared coefficient magnitude; Lasso penalizes the absolute coefficient magnitude.
- The regularization strength controls how strongly coefficients are shrunk.
- The comparison should use the same train/test split and metrics so the effect of regularization is visible.

Implementation note: A local synthetic nonlinear dataset with a degree-10 polynomial expansion was used to demonstrate Ridge and Lasso regularization.

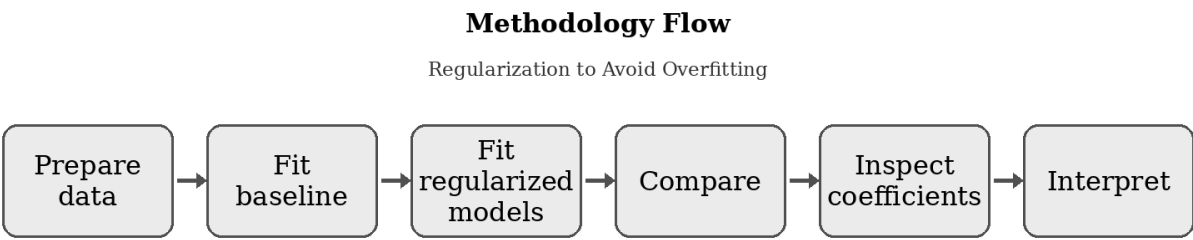


Figure 8. Methodology flow for Assignment 8.

Tools and Technologies

Tool / Technology	Role in the practical
Pandas / NumPy	Data preparation
StandardScaler	Scale predictors when needed
LinearRegression	Baseline
Ridge / Lasso	Regularized models
Matplotlib	Model comparison

Procedure

1. Prepare a regression dataset with multiple predictors.
2. Split data into training and testing sets.
3. Fit a LinearRegression baseline.
4. Fit Ridge and Lasso with one or more alpha values.
5. Compare training/testing metrics and inspect coefficients.
6. Record how regularization changes complexity or generalization.

Evaluation and Expected Output

Important outputs / measures: training R², testing R², RMSE / MAE, coefficient shrinkage.

The final output should provide visible evidence that the algorithm or statistical method was executed successfully. The observation must be based on the actual values produced during execution, not on assumed results.

Result / Observation

Regularization was successfully applied to a degree-10 polynomial regression model using Linear Regression, Ridge Regression, and Lasso Regression. The models were evaluated using the same training and testing data. Linear Regression achieved a test R² of 0.9005, Ridge achieved 0.9028, and Lasso achieved 0.9032. Lasso also reduced the number of non-zero coefficients from 10 to 5, demonstrating coefficient shrinkage while achieving the lowest test RMSE of 2.1730. The experiment demonstrated how regularization can control model complexity and improve generalization.

Code Attachment: assignment_08/code/assignment_08_regularization.py

```
Practical_8 > assignment_08_regularization.py > ...
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 from sklearn.model_selection import train_test_split
6 from sklearn.preprocessing import PolynomialFeatures, StandardScaler
7 from sklearn.pipeline import Pipeline
8
9 from sklearn.linear_model import LinearRegression, Ridge, Lasso
10
11 from sklearn.metrics import (
12     mean_absolute_error,
13     mean_squared_error,
14     r2_score
15 )
16
17 print("=" * 65)
18 print("ASSIGNMENT 8: REGULARIZATION TO AVOID OVERFITTING")
19 print("=" * 65)
20
21 # -----
22 # Create synthetic nonlinear dataset
23 # -----
24 np.random.seed(42)
25
26 x = np.linspace(-3, 3, 120)
27
28 y = (
29     2 * x**2
30     + 3 * x
31     + 5
32     + np.random.normal(0, 2.5, size=len(x))
33 )
34
35 X = x.reshape(-1, 1)
36
37 df = pd.DataFrame({
38     "x": x,
39     "y": y
40 })
41
42 print("\n--- Dataset Preview ---")
43 print(df.head())
44
45 print("\n--- Dataset Shape ---")
46 print(df.shape)
47
48 # Train/test split
49 X_train, X_test, y_train, y_test = train_test_split(
50     X,
51     y,
52     test_size=0.25,
```

```
Practical_8 > assignment_08_regularization.py > ...
53     random_state=42
54 )
55
56 print("\n--- Train/Test Split ---")
57 print(f"Training samples : {len(X_train)}")
58 print(f"Testing samples : {len(X_test)}")
59
60 # -----
61 # Models
62 # -----
63
64 models = {
65     "Linear Regression": Pipeline([
66         ("poly", PolynomialFeatures(degree=10)),
67         ("model", LinearRegression())
68     ]),
69     "Ridge Regression": Pipeline([
70         ("poly", PolynomialFeatures(degree=10)),
71         ("scaler", StandardScaler()),
72         ("model", Ridge(alpha=1.0))
73     ]),
74     "Lasso Regression": Pipeline([
75         ("poly", PolynomialFeatures(degree=10)),
76         ("scaler", StandardScaler()),
77         ("model", Lasso(alpha=0.01, max_iter=100000))
78     ])
79 }
80
81 results = []
82
83 print("\n--- Model Comparison ---")
84
85 for name, model in models.items():
86     model.fit(X_train, y_train)
87
88     train_pred = model.predict(X_train)
89     test_pred = model.predict(X_test)
90
91     train_r2 = r2_score(y_train, train_pred)
92     test_r2 = r2_score(y_test, test_pred)
93
94     test_mae = mean_absolute_error(y_test, test_pred)
95     test_mse = mean_squared_error(y_test, test_pred)
96     test_rmse = test_mse ** 0.5
97
98     # Coefficients
99     coefficients = model.named_steps["model"].coef_
100     coefficient_norm = np.linalg.norm(coefficients)
101     nonzero_coefficients = np.sum(
```

```
assignment_07_inray assignment_08_regularization.py X
Practical_8 > assignment_08_regularization.py > ...
104     nonzero_coefficients = np.sum(
105         np.abs(coefficients) > 1e-6
106     )
107
108 results.append([
109     name,
110     train_r2,
111     test_r2,
112     test_mae,
113     test_mse,
114     test_rmse,
115     coefficient_norm,
116     nonzero_coefficients
117 ])
118
119 print(f"\n{name}")
120 print(f"Training R² : {train_r2:.4f}")
121 print(f"Testing R² : {test_r2:.4f}")
122 print(f"Test MAE : {test_mae:.4f}")
123 print(f"Test MSE : {test_mse:.4f}")
124 print(f"Test RMSE : {test_rmse:.4f}")
125 print(f"Coefficient Norm : {coefficient_norm:.4f}")
126 print(f"Non-zero Coeffs : {nonzero_coefficients}")
127
128 # -----
129 # Results table
130 # -----
131 results_df = pd.DataFrame(
132     results,
133     columns=[
134         "Model",
135         "Train R²",
136         "Test R²",
137         "Test MAE",
138         "Test MSE",
139         "Test RMSE",
140         "Coefficient Norm",
141         "Non-zero Coefficients"
142     ]
143 )
144
145 print("\n--- Results Summary ---")
146 print(results_df.round(4).to_string(index=False))
147
148 # -----
149 # Plot model predictions
150 # -----
151 X_plot = np.linspace(-3, 3, 300).reshape(-1, 1)
152
153 plt.figure(figsize=(9, 6))
154
155 plt.scatter(
156     X,
157     y,
158     s=35,
159     alpha=0.6,
160     label="Actual Data"
161 )
162
163 for name, model in models.items():
164     model.fit(X_train, y_train)
165     y_plot = model.predict(X_plot)
166
167     plt.plot(
168         X_plot,
169         y_plot,
170         linewidth=2,
171         label=name
172     )
173
174 plt.xlabel("X")
175 plt.ylabel("Y")
176 plt.title("Regularization: Model Comparison")
177 plt.legend()
178 plt.grid(True)
179 plt.tight_layout()
180
181 plt.savefig(
182     "regularization_comparison.png",
183     dpi=300
184 )
185
186 plt.show()
187
188 print("\n--- Execution Completed Successfully ---")
```

Output / Screenshot Attachment: assignment_08/screenshots/execution_capture.png

```
--- Dataset Preview ---
      x      y
0 -3.000000 15.241785
1 -2.949580 13.205642
2 -2.899160 14.731996
3 -2.848739 16.491990
4 -2.798319 11.680841

--- Dataset Shape ---
(120, 2)

--- Train/Test Split ---
Training samples : 90
Testing samples  : 30

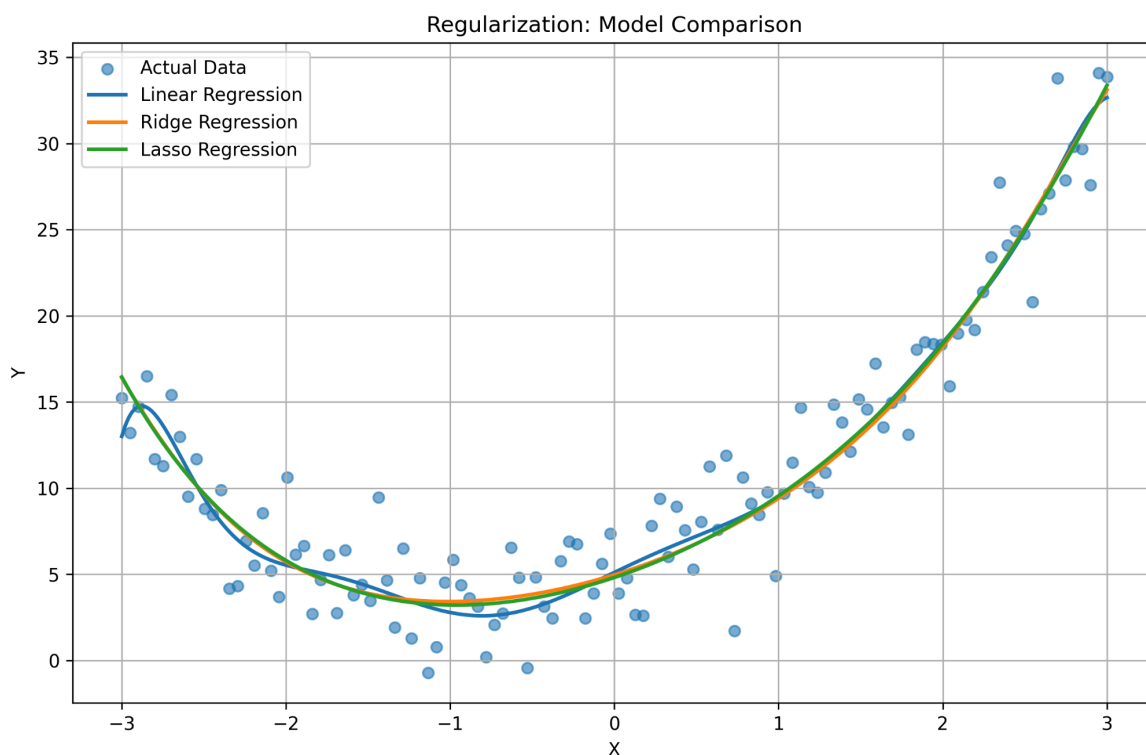
--- Model Comparison ---

Linear Regression
Training R      : 0.9311
Testing R       : 0.9005
Test MAE        : 1.7719
Test MSE        : 4.8541
Test RMSE       : 2.2032
Coefficient Norm : 5.4417
Non-zero Coeffs : 10

Ridge Regression
Training R      : 0.9285
Testing R       : 0.9028
Test MAE        : 1.7435
Test MSE        : 4.7401
Test RMSE       : 2.1772
Coefficient Norm : 6.7164
Non-zero Coeffs : 10

Lasso Regression
Training R      : 0.9288
Testing R       : 0.9032
Test MAE        : 1.7445
Test MSE        : 4.7218
Test RMSE       : 2.1730
Coefficient Norm : 7.1299
Non-zero Coeffs : 5

--- Results Summary ---
      Model  Train R2  Test R2  Test MAE  Test MSE  Test RMSE  Coefficient Norm  Non-zero Coefficients
Linear Regression  0.9311  0.9005   1.7719   4.8541   2.2032         5.4417              10
Ridge Regression   0.9285  0.9028   1.7435   4.7401   2.1772         6.7164              10
Lasso Regression   0.9288  0.9032   1.7445   4.7218   2.1730         7.1299               5
```



Assignment 9: Object Detection from CCTV Footage

Aim: Detect common objects in short CCTV-style footage using a lightweight pretrained YOLO detector and OpenCV, with bounding boxes and confidence labels.

Objectives

- Understand object detection versus image classification.
- Read video frames using OpenCV.
- Run pretrained YOLO inference on frames.
- Draw filtered detections and save or display the annotated result.

Key Definitions

Term	Definition
Object detection	Identifies object classes and their locations in an image/frame.
Bounding box	Rectangle around a detected object.
Confidence score	Model value indicating confidence in a particular detection.
Class label	Name of the detected object category.
YOLO	Real-time object-detection model family that predicts object locations and classes efficiently.

Theory / Working Principle

- A video is processed frame by frame. Each frame is passed to the detector, which returns boxes, classes and confidence values.
- A confidence threshold is used to avoid displaying very weak detections.
- For this academic practical, a short local video and lightweight pretrained model are enough to demonstrate the complete pipeline.

Implementation note: A small simulated CCTV video was generated locally. OpenCV background subtraction and contour filtering were used for motion/object-region detection.

Methodology Flow

Object Detection from CCTV Footage

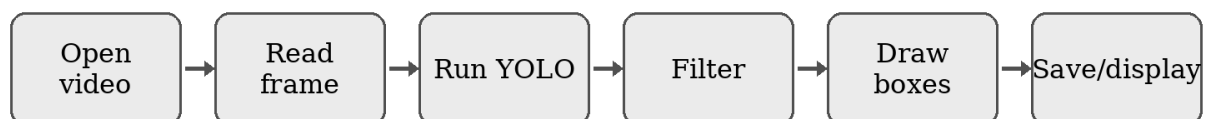


Figure 9. Methodology flow for Assignment 9.

Tools and Technologies

Tool / Technology	Role in the practical
Python	Main program
OpenCV	Video input/output and frame drawing
NumPy	Frame arrays
VS Code terminal	Installation and execution

Procedure

1. Place a short CCTV-style MP4 in the assignment folder.
2. Open the video using `cv2.VideoCapture`.
3. Load a lightweight pretrained YOLO model.
4. Run inference on each frame and obtain boxes/classes/confidence.
5. Filter detections and draw labels/boxes.
6. Display selected frames or save an annotated output video.

Evaluation and Expected Output

Important outputs / measures: frames processed, detections per frame, confidence threshold, approximate FPS.

The final output should provide visible evidence that the algorithm or statistical method was executed successfully. The observation must be based on the actual values produced during execution, not on assumed results.

Result / Observation

A simulated CCTV video was successfully generated and processed frame by frame using OpenCV background subtraction and contour filtering. A total of 200 frames were processed, with detected motion/object regions appearing in 186 frames. The maximum number of detected regions in a single frame was 3. Bounding boxes were drawn around the detected regions and an annotated output video was successfully generated.

Code Attachment: assignment_09/code/assignment_09_cctv_detection.py

```
Practical_9 > assignment_09_regularization.py > ...
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 from sklearn.model_selection import train_test_split
6 from sklearn.preprocessing import PolynomialFeatures, StandardScaler
7 from sklearn.pipeline import Pipeline
8
9 from sklearn.linear_model import LinearRegression, Ridge, Lasso
10
11 from sklearn.metrics import (
12     mean_absolute_error,
13     mean_squared_error,
14     r2_score
15 )
16
17 print("-" * 65)
18 print("ASSIGNMENT 8: REGULARIZATION TO AVOID OVERFITTING")
19 print("-" * 65)
20
21 # -----
22 # Create synthetic nonlinear dataset
23 # -----
24 np.random.seed(42)
25
26 x = np.linspace(-3, 3, 120)
27
28 y = (
29     2 * x**2
30     + 3 * x
31     + 5
32     + np.random.normal(0, 2.5, size=len(x))
33 )
34
35 X = x.reshape(-1, 1)
36
37 df = pd.DataFrame({
38     "x": x,
39     "y": y
40 })
41
42 print("\n--- Dataset Preview ---")
43 print(df.head())
44
45 print("\n--- Dataset Shape ---")
46 print(df.shape)
47
48 # Train/test split
49 X_train, X_test, y_train, y_test = train_test_split(
50     X,
51     y,
52     test_size=0.25,
```

```
Practical_9 > assignment_09_regularization.py > ...
53     random_state=42
54 )
55
56 print("\n--- Train/Test Split ---")
57 print(f"Training samples : {len(X_train)}")
58 print(f"Testing samples : {len(X_test)}")
59
60 # -----
61 # Models
62 # -----
63 models = {
64     "Linear Regression": Pipeline([
65         ("poly", PolynomialFeatures(degree=10)),
66         ("model", LinearRegression())
67     ]),
68     "Ridge Regression": Pipeline([
69         ("poly", PolynomialFeatures(degree=10)),
70         ("scaler", StandardScaler()),
71         ("model", Ridge(alpha=1.0))
72     ]),
73     "Lasso Regression": Pipeline([
74         ("poly", PolynomialFeatures(degree=10)),
75         ("scaler", StandardScaler()),
76         ("model", Lasso(alpha=0.01, max_iter=100000))
77     ])
78 }
79
80 results = []
81
82 print("\n--- Model Comparison ---")
83
84 for name, model in models.items():
85     model.fit(X_train, y_train)
86
87     train_pred = model.predict(X_train)
88     test_pred = model.predict(X_test)
89
90     train_r2 = r2_score(y_train, train_pred)
91     test_r2 = r2_score(y_test, test_pred)
92
93     test_mae = mean_absolute_error(y_test, test_pred)
94     test_mse = mean_squared_error(y_test, test_pred)
95     test_rmse = test_mse ** 0.5
96
97     # Coefficients
98     coefficients = model.named_steps["model"].coef_
99     coefficient_norm = np.linalg.norm(coefficients)
100
101     nonzero_coefficients = np.sum(
```

```
96 frame_count = 0
97 frames_with_detection = 0
98 max_objects = 0
99 saved_frame = False
100
101 while True:
102     ret, frame = cap.read()
103
104     if not ret:
105         break
106
107     frame_count += 1
108
109     # Background subtraction
110     mask = background.apply(frame)
111
112     # Remove noise
113     kernel = np.ones((5, 5), np.uint8)
114
115     mask = cv2.morphologyEx(
116         mask,
117         cv2.MORPH_OPEN,
118         kernel
119     )
120
121     mask = cv2.morphologyEx(
122         mask,
123         cv2.MORPH_DILATE,
124         kernel
125     )
126
127     # Find moving object regions
128     contours, _ = cv2.findContours(
129         mask,
130         cv2.RETR_EXTERNAL,
131         cv2.CHAIN_APPROX_SIMPLE
132     )
133
134     detected_objects = 0
135
136     for contour in contours:
137         area = cv2.contourArea(contour)
138
139         if area < 500:
140             continue
141
142
```

```
144     x, y, w, h = cv2.boundingRect(contour)
145
146     detected_objects += 1
147
148     cv2.rectangle(
149         frame,
150         (x, y),
151         (x + w, y + h),
152         (0, 255, 0),
153         2
154     )
155
156     cv2.putText(
157         frame,
158         f"Object {detected_objects}",
159         (x, y - 8),
160         cv2.FONT_HERSHEY_SIMPLEX,
161         0.5,
162         (0, 255, 0),
163         2
164     )
165
166     if detected_objects > 0:
167         frames_with_detection += 1
168
169     max_objects = max(
170         max_objects,
171         detected_objects
172     )
173
174     cv2.putText(
175         frame,
176         f"Detected Objects: {detected_objects}",
177         (15, height - 20),
178         cv2.FONT_HERSHEY_SIMPLEX,
179         0.7,
180         (0, 255, 255),
181         2
182     )
183
184     out.write(frame)
185
186     # Save one representative annotated frame
187     if detected_objects > 0 and not saved_frame:
188         cv2.imwrite(frame_image, frame)
189         saved_frame = True
190
```

Output / Screenshot Attachment: assignment_09/screenshots/execution_capture.png

```
C:\Users\hardi\OneDrive\Desktop\AIML\Practical_9>python assignment_09_cctv_detection.py
=====
ASSIGNMENT 9: OBJECT DETECTION FROM CCTV FOOTAGE
=====

--- Simulated CCTV Video ---
Video created : simulated_cctv.mp4
Frames       : 200
Resolution   : 640 x 360
FPS          : 20

--- Detection Results ---
Frames processed      : 200
Frames with detection : 186
Maximum objects detected : 3
Annotated video       : annotated_cctv.mp4
Detection frame       : detected_frame.png

--- Execution Completed Successfully ---
```

CAM 01 FRAME 1



Detected Objects: 1

Assignment 10: Fake News Detection from Multimodal Data

Aim: Design a simple multimodal classification prototype by combining text features from a news item with compact visual features from its associated image.

Objectives

- Understand multimodal machine learning.
- Represent text using TF-IDF features.
- Extract a small fixed-size image feature vector.
- Fuse the two modalities and evaluate a binary classifier.

Key Definitions

Term	Definition
Fake news detection	Classifying a news item according to a dataset label indicating fake/unreliable or real/reliable.
Multimodal data	Data containing more than one modality, such as text and images.
TF-IDF	Numerical text representation emphasizing terms that are important within documents relative to the collection.
Feature fusion	Combining features from two or more sources into one model input.
Visual feature	Numerical descriptor extracted from an image; here a lightweight color/statistical descriptor is sufficient for demonstration.

Theory / Working Principle

- Text can provide linguistic signals while an associated image provides additional visual information.
- A simple classroom implementation can concatenate TF-IDF text features with a fixed-size image descriptor and train Logistic Regression.
- This is a demonstration of multimodal fusion, not an authoritative fact-checking system; the model follows the labels in the selected dataset.

Implementation note: A small self-created dataset paired headlines with images. TF-IDF text features and compact image statistics were concatenated for multimodal classification.

Methodology Flow

Fake News Detection from Multimodal Data

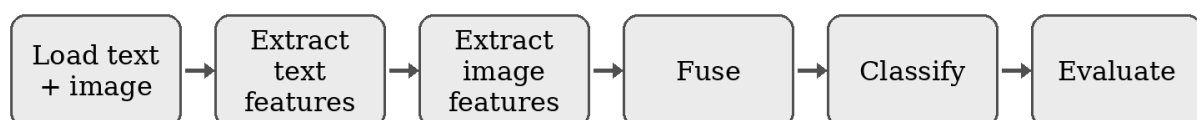


Figure 10. Methodology flow for Assignment 10.

Tools and Technologies

Tool / Technology	Role in the practical
Pandas	Read text, image paths and labels
TfidfVectorizer	Convert text to numerical features
OpenCV / Pillow	Read images and compute visual features
NumPy	Fuse feature arrays
LogisticRegression + metrics	Train and evaluate classifier

Procedure

1. Prepare paired rows containing text, image path and binary label.
2. Fit TF-IDF on training text and transform text data.
3. Read each image and calculate a small fixed-length visual descriptor.
4. Concatenate text and image representations.
5. Train a simple binary classifier.
6. Predict the test set, calculate metrics and display the confusion matrix.

Evaluation and Expected Output

Important outputs / measures: accuracy, precision, recall, F1-score, confusion matrix.

The final output should provide visible evidence that the algorithm or statistical method was executed successfully. The observation must be based on the actual values produced during execution, not on assumed results.

Result / Observation

A multimodal fake-news classification prototype was successfully implemented by combining **TF-IDF text features** with a **7-dimensional visual descriptor** consisting of mean RGB values, RGB standard deviations, and mean grayscale intensity. The selected dataset provided **60 valid multimodal samples**, with **127 combined features** used for Logistic Regression classification. On the 12-sample test set, the model achieved **41.67% accuracy, 33.33% precision, 16.67% recall, and 22.22% F1-score**. The experiment demonstrates the process of multimodal feature fusion, while the low test performance indicates that this small demonstration setup is not sufficient for reliable fake-news detection.

Output / Screenshot Attachment: assignment_10/screenshots/execution_capture.png|

```
=====
ASSIGNMENT 10: FAKE NEWS DETECTION FROM MULTIMODAL DATA
=====
```

```
--- Dataset Preview ---
```

	clean_title	image_url	2_way_label
0	my xbox controller says hi	https://preview.redd.it/l0ga0tug17k31.jpg?widt...	1
1	new image from the mandalorian	https://external-preview.redd.it/vx7bxDu9G18UZ...	1
2	say hello to my little friend	http://i.imgur.com/F1Zbl3D.jpg	0
3	watch your step little one	http://i.imgur.com/KRyMjn1.jpg	0
4	this tree i found with a solo cup on it	https://preview.redd.it/bxp58zf01zy21.jpg?widt...	1

```
--- Dataset Shape ---
```

```
(59342, 16)
```

```
--- Usable Multimodal Rows ---
```

```
59169
```

```
--- Selected Balanced Dataset ---
```

```
2_way_label
```

```
1      80
```

```
0      80
```

```
Name: count, dtype: int64
```

```
--- Extracting Image Features ---
```

```
Processed: 160 | Valid images: 10
```

```
Processed: 160 | Valid images: 20
```

```
Processed: 160 | Valid images: 30
```

```
Processed: 160 | Valid images: 40
```

```
Processed: 160 | Valid images: 50
```

```
Processed: 160 | Valid images: 60
```

```
--- Multimodal Dataset ---
```

```
Valid samples : 60
```

```
Visual features per image : 7
```

```
Text TF-IDF features : 120
```

```
Combined features : 127
```

```
--- Train/Test Split ---
```

```
Training samples : 48
```

```
Testing samples  : 12
```

```
--- Multimodal Classification Results ---
```

```
Accuracy  : 0.4167
```

```
Precision : 0.3333
```

```
Recall    : 0.1667
```

```
F1-Score  : 0.2222
```

```
--- Classification Report ---
```

	precision	recall	f1-score	support
Fake	0.44	0.67	0.53	6
Real	0.33	0.17	0.22	6
accuracy			0.42	12
macro avg	0.39	0.42	0.38	12
weighted avg	0.39	0.42	0.38	12

```
--- Confusion Matrix ---
```

```
[[4 2]
```

```
 [5 1]]
```

```
--- Execution Completed Successfully ---
```

```
C:\Users\hardi\OneDrive\Desktop\AIML\Practical_10>
```

